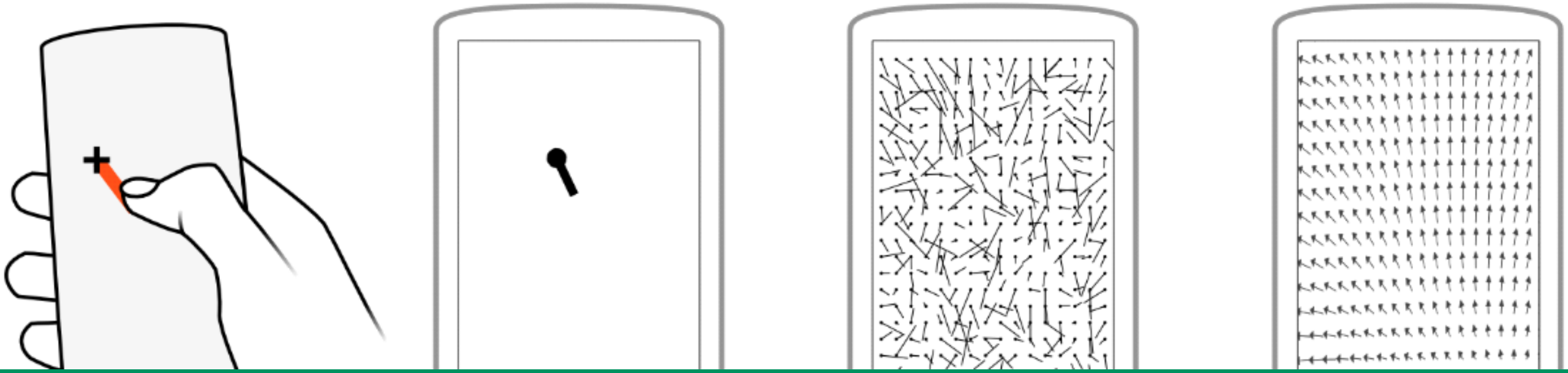
UNIVERSITÄT BAYREUTH

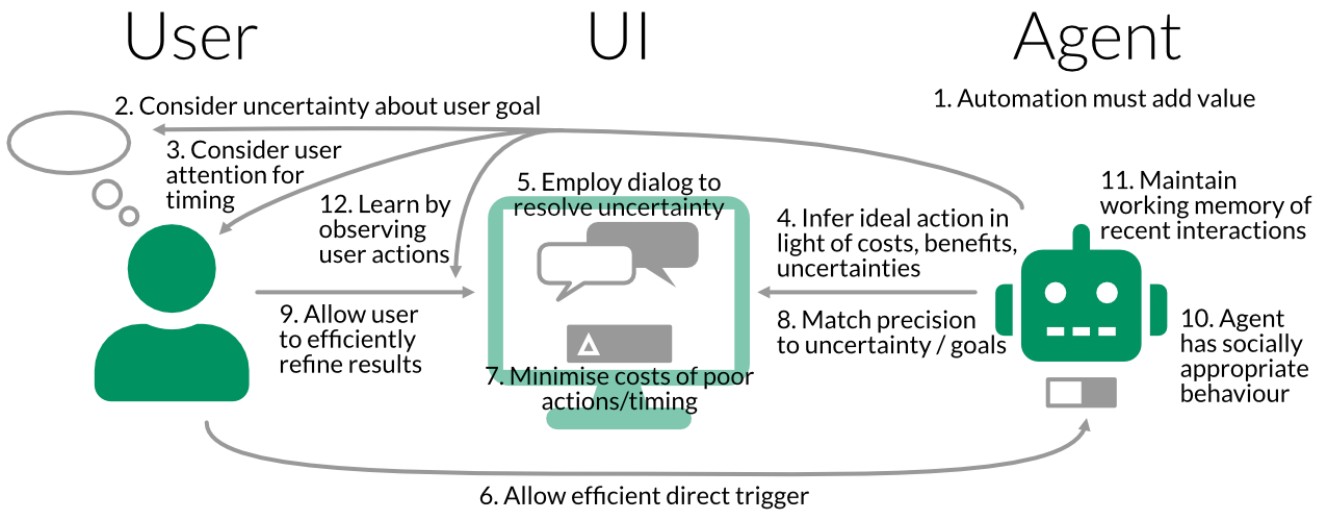


User Modelling & Adaptive UIs

Recap & Last Quiz

Lecture Quiz

Think about **Horvitz' twelve principles** in the context of the design of the **word suggestion feature on your smartphone keyboard:** To what extent are (some of) these principles realised? And could you use them to redesign or improve the word suggestion feature in some way?



✅ **Automation must add value:** Adds value by correcting spelling and shortening [manual] typing

✅ **Allow efficient direct trigger:** The user triggers the [display of the] suggestions simply by typing. The interaction requires no additional action. [only for accepting one]

✅ **Learn by observing user actions:**
Two interpretations
- Over long-term use: suggestions improve with typing history
- In the moment: continually updating the suggestions [as the user types more of a word]

✅ **Consider uncertainty about user goal:** The main way that autocomplete seems to account for uncertainty is by offering multiple completion suggestions, in the hopes that one of them is the relevant/desired one.

✅ **Maintain working memory of recent interactions:** Keyboards can learn from recent messages or patterns from the user

Lecture Quiz

Think about **Horvitz' twelve principles** in the context of the design of the **word suggestion feature on your smartphone keyboard:** To what extent are (some of) these principles realised? And could you use them to redesign or improve the word suggestion feature in some way?

❌ **Consider user attention for timing:** For me, it is often just much easier to type the whole thing rather than wait for the autocomplete suggestion, select it and then go on. It also seems to interrupt the flow of typing.

❌ **Allow user to efficiently refine result:** User isn't able to refine these result (at least proactive maybe through indirect feedback or behavior)

❌ **Dialogue is not employed** to resolve conflicts [uncertainty].

❌ **Minimise Costs of Poor Actions or Timing:** suggestions popping up while typing quickly can lead to accidental taps. To address this, the system could wait to show suggestions until there's a brief pause in typing, or display them in a more subtle way until the user confirms with a swipe. Adding an easy “undo” option would also help reduce the impact of unintended choices.

Lecture Quiz

Ideas for improvements / extensions:

💡 **Various "personalities" that can be selected.** For example, the keyboard could learn to suggest more formal language when writing an email to a boss versus informal slang when messaging a friend. It could also learn to avoid suggesting certain words in specific conversational contexts.

💡 **"app awareness",** where the suggestion style differs depending on which app I am in.

💡 **User can give quick feedback** (“not this word” or “more like this”) and it could improve system learning and match user intent better.

💡 **Automatic or manual mode that changes given what the user wants.** When writing an application or a CV it should offer more words and suggestions regarding that, when writing your friends it should largely switch its vocabulary.

💡 Maybe letting the user proactively give the suggestion system suggestions. This could help the system to give the right suggestions. Could be described as an **onboarding process** for the system.

If you're curious…

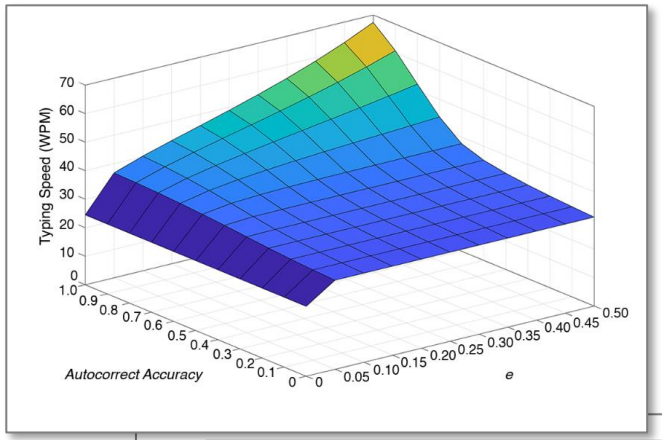
About suggestions and auto-correct

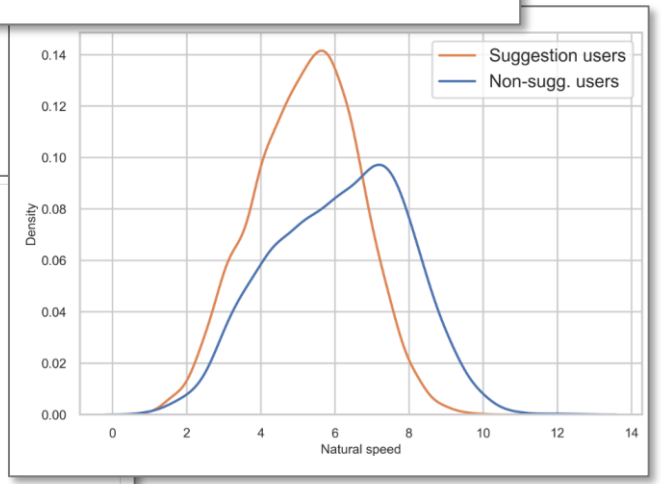
- What do you think is the **"role" of text suggestions** for your writing process?
  Interacting with Next-Phrase Suggestions: How Suggestion Systems Aid and Influence the Cognitive Processes of Writing

- **Timing** of suggestions – is it a good idea to always show suggestions?
  A Cost-Benefit Study of Text Entry Suggestion Interaction

- **Autocorrect** – could you benefit from relying more on it?
  The Limits of Expert Text Entry Speed on Mobile Keyboards with Autocorrect

- Word **suggestions can slow you down** – why are they still used?
  Typing Behavior is About More than Speed: Users' Strategies for Choosing Word Suggestions Despite Slower Typing Rates





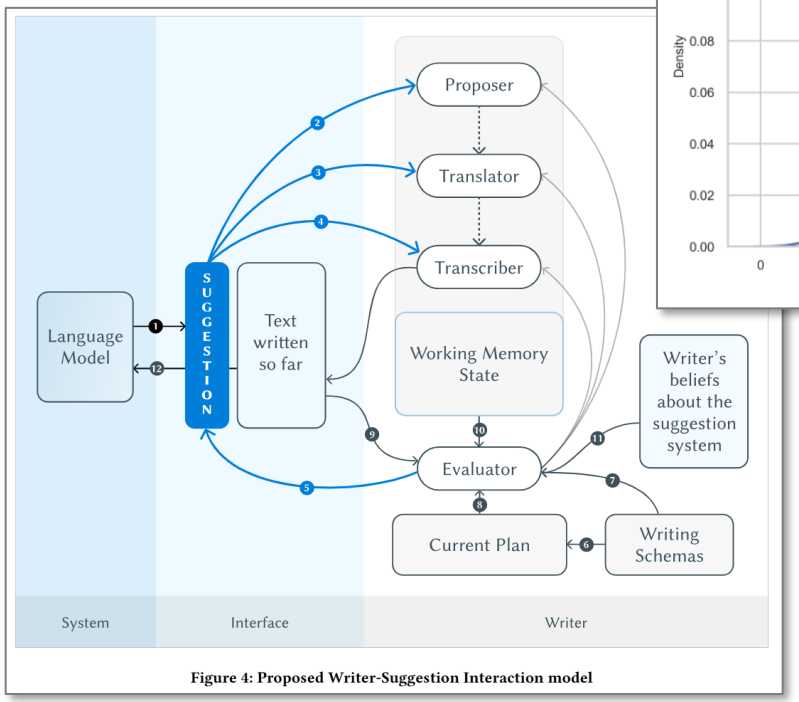


Figure 4: Proposed Writer-Suggestion Interaction model

Lecture Quiz

Apply the framework by Guzdial & Riedl to examine the mood board system by Koch et al.

Aspect	User	System
Start	Adds first image to the moodboard	None (or: suggests image, if we assume it can suggest sth for an empty board)
Artifact actions	Adding/editing images on the board	None (the system cannot directly edit the board)
Other actions	Trigger new recommendations (i.e. the 3 buttons)	Recommends an image (+ explanation)
Non-turn actions	Not so clear, possibly: Read explanation	Learn from user actions
Turns	End after pressing one of the buttons or adding a picture (assuming that the AI updates its recommendation after each board change)	End after displaying a new suggestion
End	User decides (e.g. satisfied with board)	-

This lecture

Learning goals

After this session, you should be able to:

- ✓ **Explain** the motivation and key concepts of adaptive UIs; with motivating examples
- ✓ **Analyse** a given UI adaptation idea to recommend a simple modelling approach (for cases similar to what we cover in the case studies here)
- ✓ **Explain** Fitts' Law
- ✓ **Apply** a probabilistic perspective on interaction: E.g. what contributes uncertainty in a particular case of UI adaptation? How might we handle this in the UI and/or in the model?

Intro

Why, What, When, How?

Share your thoughts

Let‘s collect some thoughts on:

- **Why** might we want to adapt a UI or interactive system to a (specific) user?
- **How** might we do this? (conceptually and/or technically)

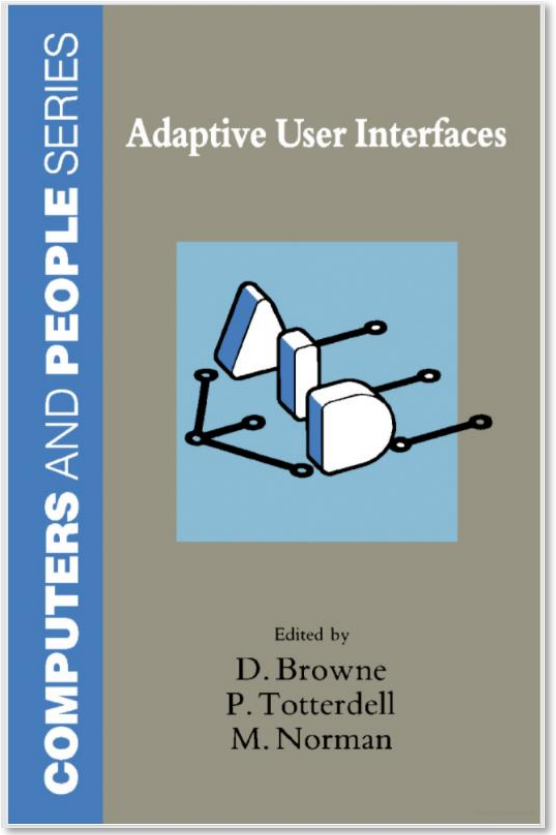
Origins of UI adaptation

As a way towards “computers that understand humans”

It is widely believed that "everyone should be computer literate" and as a consequence vast human and financial resources are being expended internationally on training individuals to adapt to using computers within their lives.

The premise of the research that is reported in this volume is that ***"computers should be user literate"****.*

- Browne, Totterdell, Norman 1990



More concretely: Why adapt a UI?

Purpose of adaptations

- Extend system‘s lifespan
- Widen user base
- Enable user goals
- Satisfy user needs
- Improve operational accuracy
- Increase operational speed
- Reduce operational learning
- Enhance user understanding

} Focus of today‘s lecture

Underlying **assumption:**
There are differences between users that are relevant to using the system.

[Browne et al., 1990]

What might we consider in a user model?

Aspects that might vary between users/contexts:

- Interaction behaviour (e.g. speed, accuracy) ← This lecture
- Preferences (e.g. content-related) ← Recommender systems + this lecture
- Strategies (e.g. decision making)
- Physical aspects (e.g. hand size)
- Skills/expertise (e.g. language proficiency)
- ID (i.e. „who?“ - user identification)
- Temporal and temporary changes (e.g. ageing, broken arm)
- Context-specific aspects (e.g. influence of walking)
- …

When to adapt?

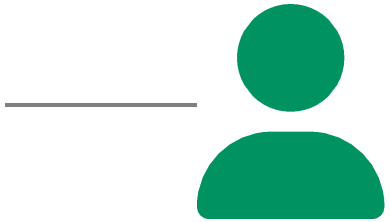
Timescales of user modelling and UI adaptation

Single action
(e.g. one touch)

Many actions
(e.g. typing behaviour
across many keystrokes)

Repeating patterns / habits / preferences
(e.g. chatting with a friend every day/weekend, …)

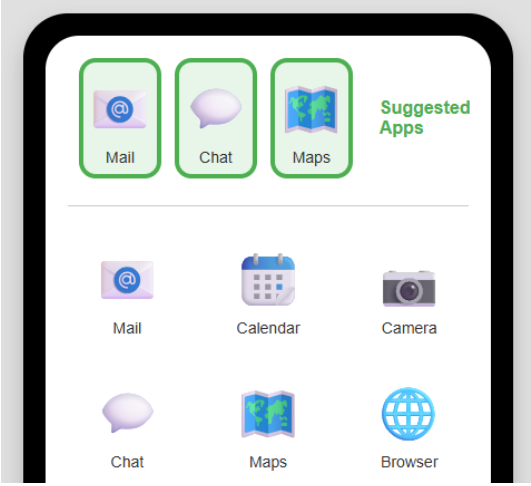
Stable / unrelated to specific interaction
(e.g. physical and psychological aspects)



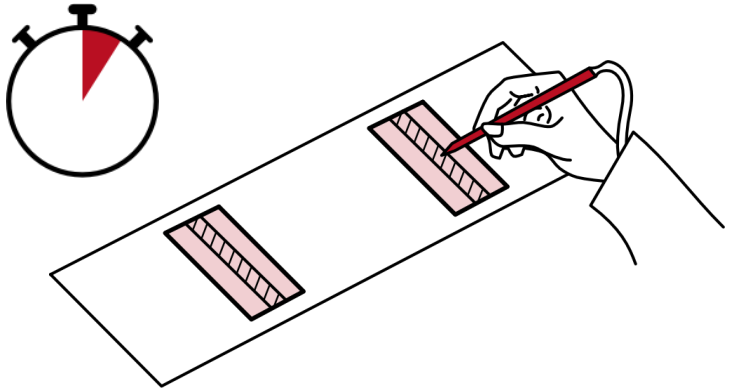
This session

Learning approach

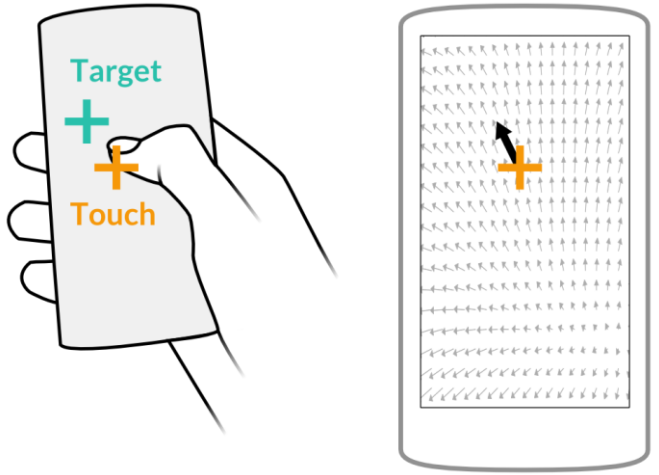
Three case studies:



App prediction



Pointing time (Fitts' Law)

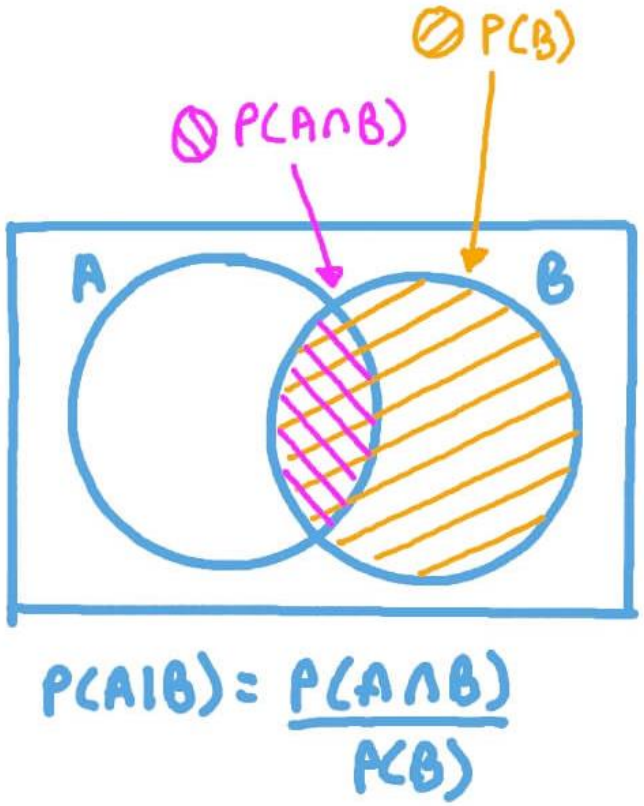


Touch targeting

Modelling behaviour (choice, time, space)
and **connecting the models to UI** features / analysis / adaptation

Good to know

Probability notation, conditional probability



Case study 1: App prediction

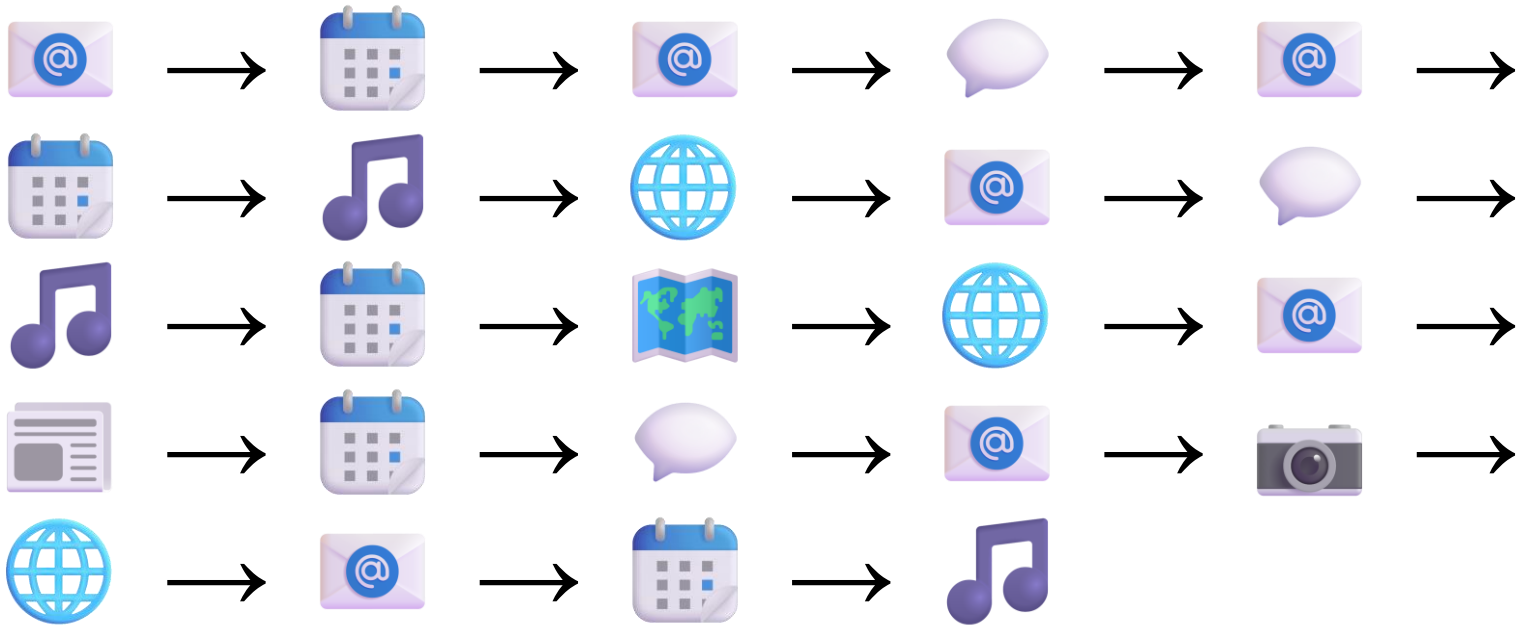
Data: User's app interaction sequences

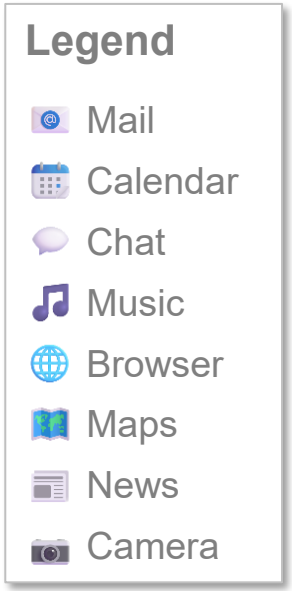
Model: Markov chain

Application: Adaptive app home screen

Example interaction data

Let's assume we have logged this sequence of used apps





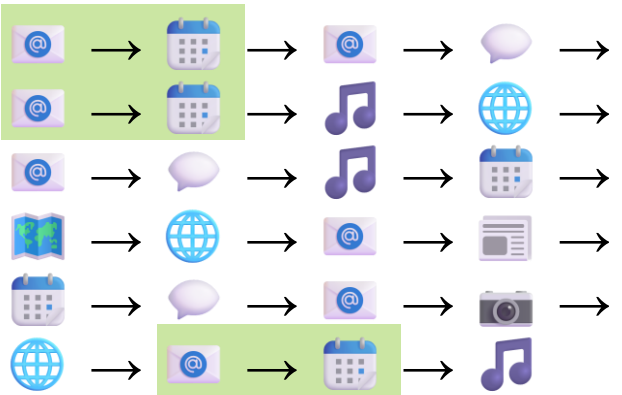
Counting transitions

Example

From ↓ / to →	Browser	Calendar	Camera	Chat	Mail	Maps	Music	News
Browser					3			
Calendar				1	1	1	2	
Camera	1							
Chat					2		1	
Mail		3	1	2				1
Maps	1							
Music	1	1						
News		1						

Note: Empty cells are 0

Example:
Counting occurrences of the transition of “mail to calendar” in our sequence



Normalizing transition counts

Divide each row by the sum of its entries

From ↓ / to →	Browser	Calendar	Camera	Chat	Mail	Maps	Music	News	SUM
Browser					3				3
Calendar				1	1	1	2		5
Camera	1								1
Chat					2		1		3
Mail		3	1	2				1	7
Maps	1								1
Music	1	1							2
News		1							1

Note: Empty cells are 0

Transition probabilities

Sum up to 1 per row

From ↓ / to →	Browser	Calendar	Camera	Chat	Mail	Maps	Music	News
Browser					1			
Calendar				.20	.20	.20	.40	
Camera	1							
Chat					.67		.33	
Mail		.43	.14	.29				.14
Maps	1							
Music	.50	.50						
News		1						

This is a **Markov chain model**

The **next state** (at time t+1) **depends only on the previous state** (at time t); i.e. we model $p(app_{t+1} | app_t)$

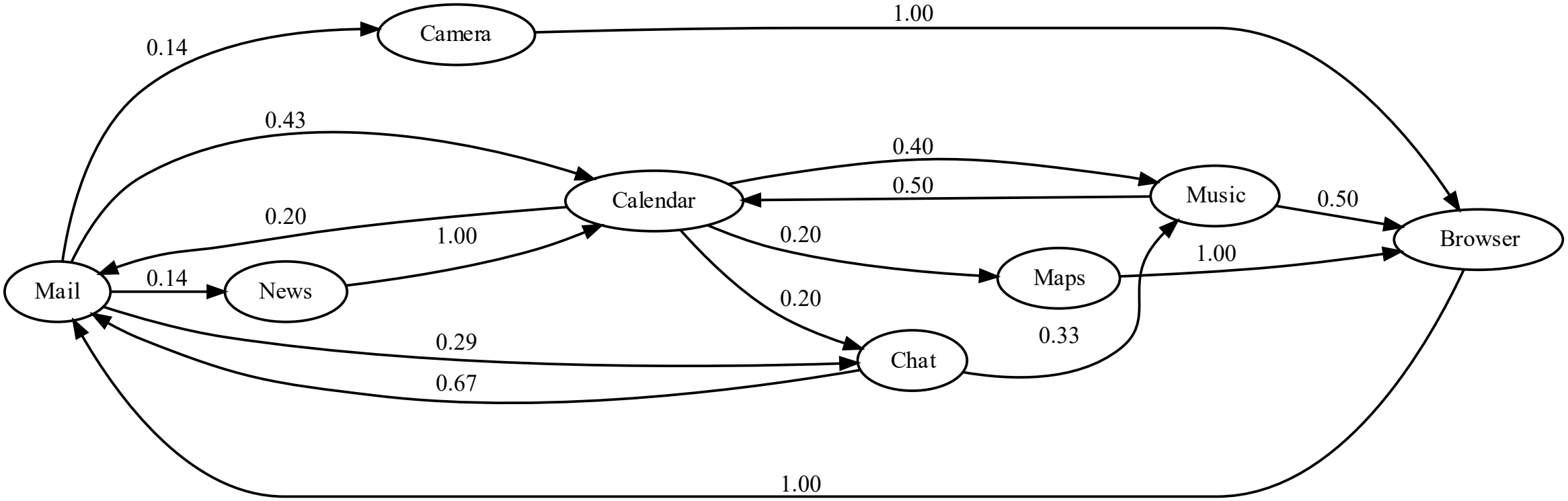
We trained (or "fitted") this model on our dataset (counting & normalizing).

Now, the model is described by this **transition matrix** $\boldsymbol{M}$. Each entry M_{ij} is our estimate of $p(app_j | app_i)$.

This is an "estimate" (of the "true" probability) because we estimated it from our limited dataset.

Our Markov chain model

Visualized as a graph / node-link diagram



Using the model

Making predictions

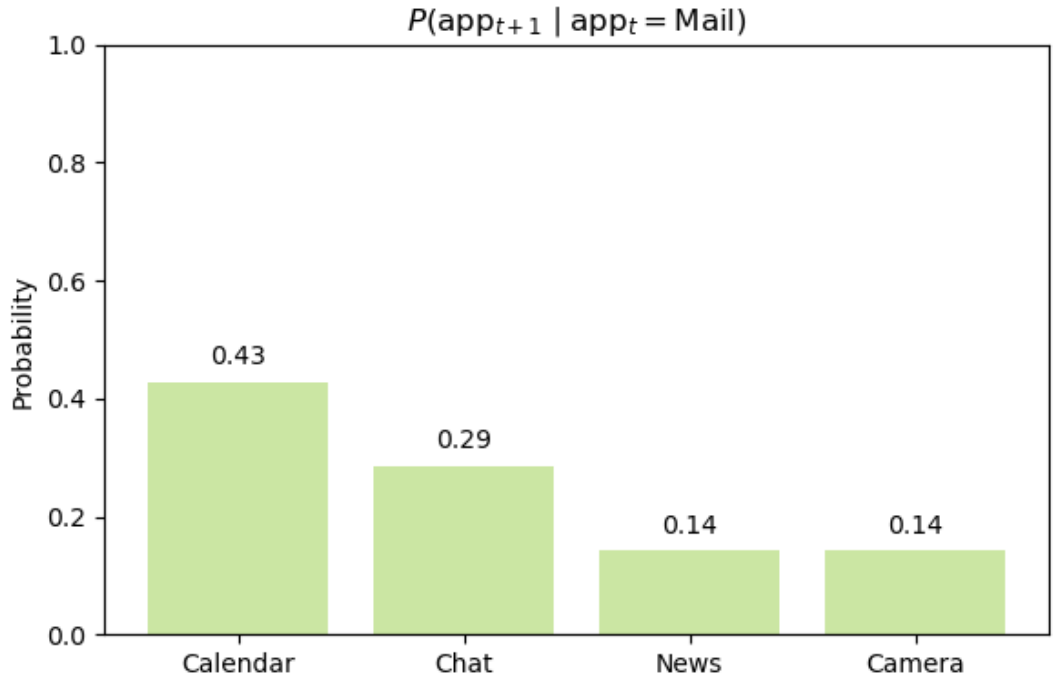
To make **predictions**, we use the distribution $p(app_{t+1} \mid app_t)$, for the last opened app app_t

In our implementation as a transition matrix, this means looking up the **row** for app_t:

	Browser	Calendar	Camera	Chat	Mail	Maps	Music	News
Browser					1			
Calendar				.20	.20	.20	.40	
Camera	1							
Chat					.67		.33	
Mail		.43	.14	.29				.14
Maps	1							
Music	.50	.50						
News		1						

E.g. for $app_t =$  , the distribution $p(app \mid$  $)$ looks like this →

Q: What are some UI features that we could build with this distribution?



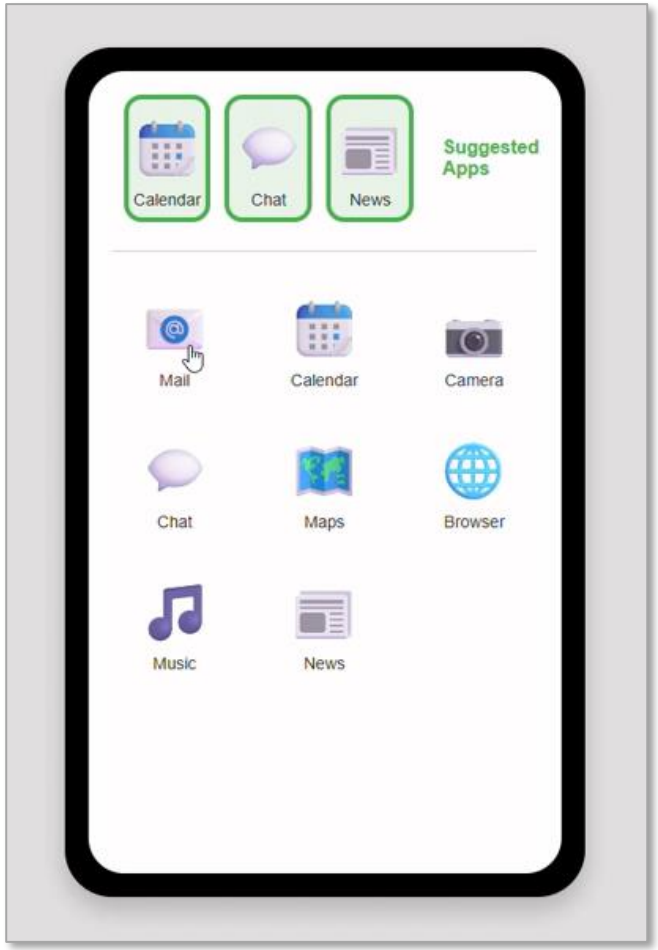
Using the model

Adapting the UI based on the predictions

We can use this distribution for various UI features, e.g.:

- Show the **most likely** app argmax($p(app \mid$ ))
- Show the N **most likely** apps
- **Sample** an app from this distribution

Shows the top 3 apps



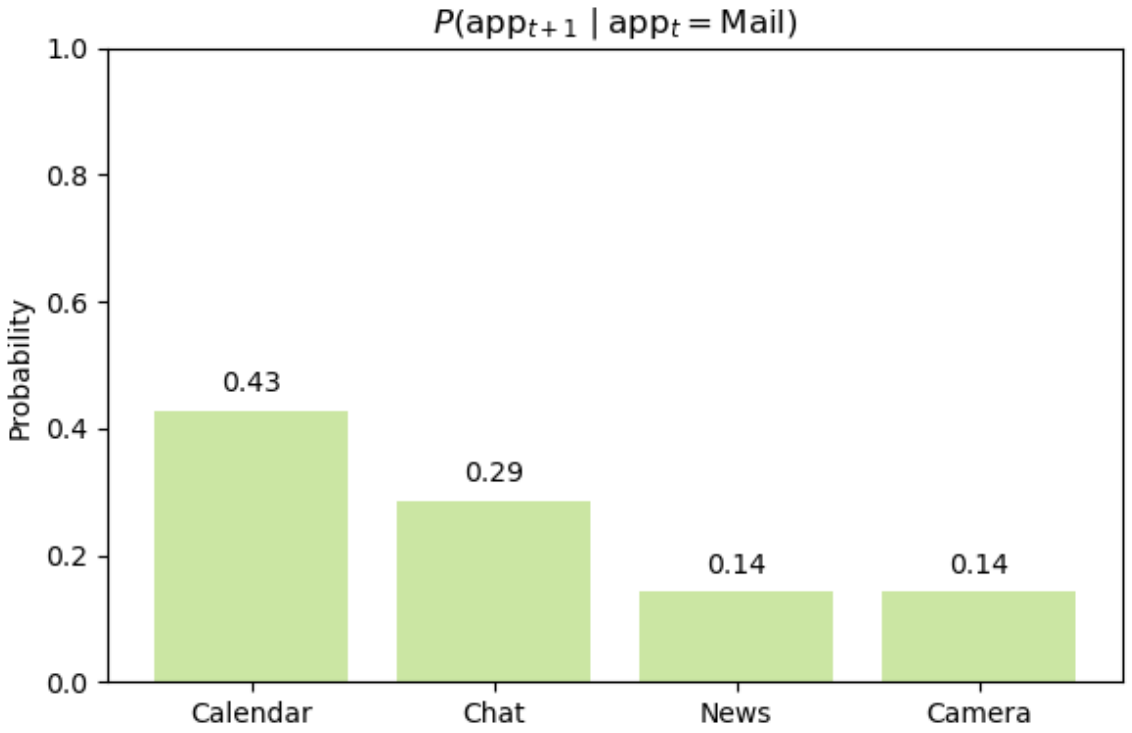
Interactive demo available in the notebook

Extending the model

What might we add to improve our predictions?

Maybe the current app is not always enough info?

E.g. News and Camera seem equally likely in our example:



Short discussion:
Which other information could we include for app prediction? How?

🤔

Extending the model

Adding time

- Extension: Let's consider **time of day**
- A (naïve) approach:
  - Build a model for $\boldsymbol{p(app_{t+1} \mid hour)}$
  - Combine that with our existing model

$$p(app_{t+1} \mid app_t) \cdot p(app_{t+1} \mid hour)$$

- Alternative:
  Directly model $p(app_{t+1} \mid app_t, hour)$, which might require more data (to observe enough combinations of apps and hours).

Notes:

We can think of this joint model as **both sources of evidence (current app & hour) "voting"** on the next app.

This is **simplifying** in the sense that the true joint distribution may contain interactions (e.g., maybe "Calendar → Music" is more likely at 6 PM than 10 AM).

But: This simplification **can also be a benefit!** If we have not observed a specific transition, we can still "back off" to using the hour-based term alone (assuming that the app-based term is never set to exactly 0; e.g. in a practical implementation, we could initialize all transitions with a very small value like 0.0001).

Extending the model

Adding time of day

- Example: **Multinomial logistic regression**
  to predict probability of each app being used, given hour of day

- **Assumptions**
  - Apps are "**competitive**":
    Across apps, the probabilities at any specific point in time add up to 1.
  - **Each app peaks only once** per day:
    Can only model one "hump" per app, unless we give it nonlinear features

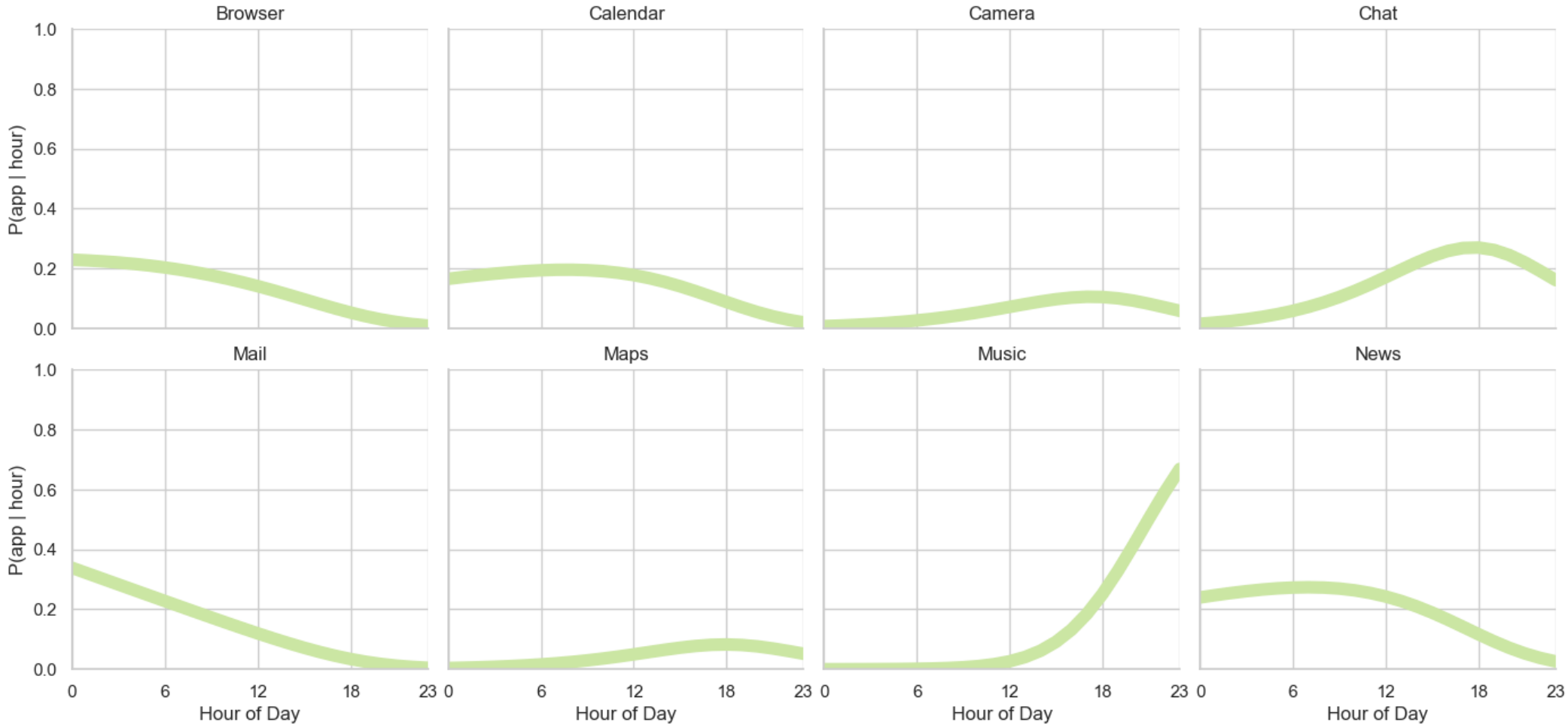
- **Alternative**
  - Logistic regression treats time as continuous
  - If we treat time as discrete (e.g. one "bin" per hour"), we could again estimate simply by counting & normalizing as above (e.g. "Mail" might have a 50% share in the 9-10am bin)

More info on this model:

- 8-minute intro to logistic regression: YouTube
- Lecture: YouTube
- Code: scikit-learn
- Background: Wikipedia

Example

A multinomial log. reg. model fitted on an extended version of our dataset



Using the extended model

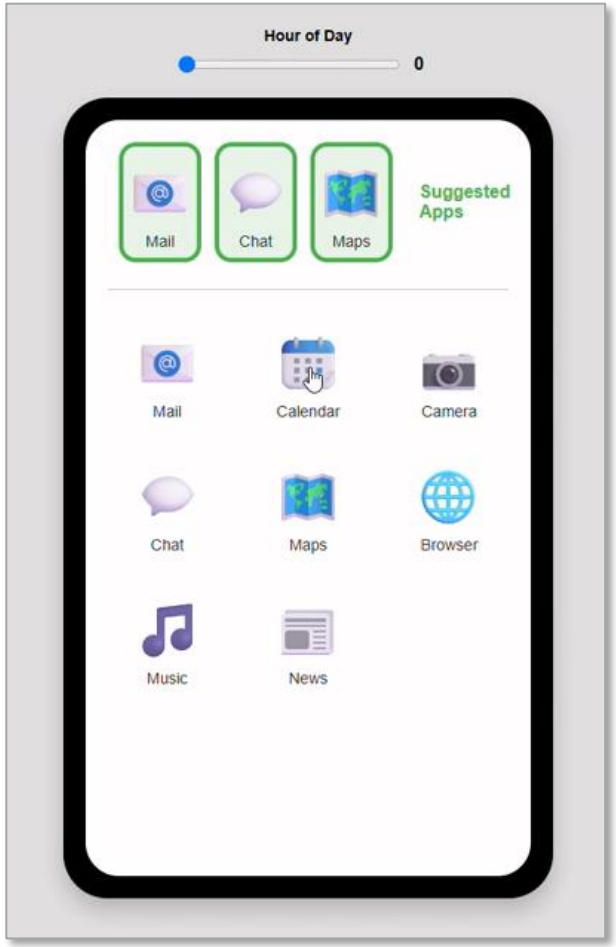
Adapting the UI based on the combined predictions

Let's explore our new model

$$p(app_{t+1} \mid app_t) \cdot p(app_{t+1} \mid hour)$$

by changing the **time**
while keeping the **current app** constant.

Selecting "Calendar" and changing the slider to see what impact time has on the prediction

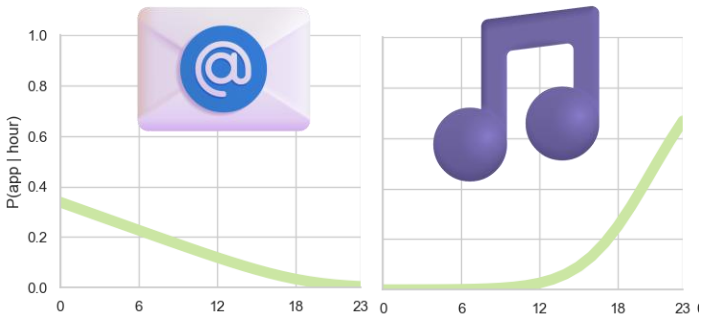


Interactive demo available in the notebook

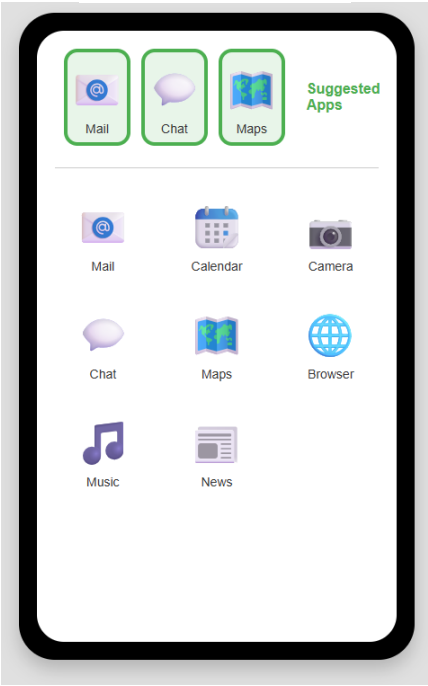
Example

Calendar: Mail now preferred over Music earlier in the day

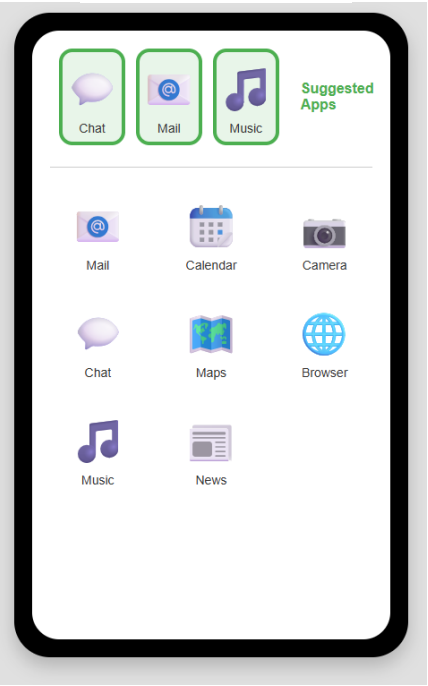
From ↓ / to →	Browser	Calendar	Camera	Chat	Mail	Maps	Music	News
Calendar				.20	.20	.20	.40	



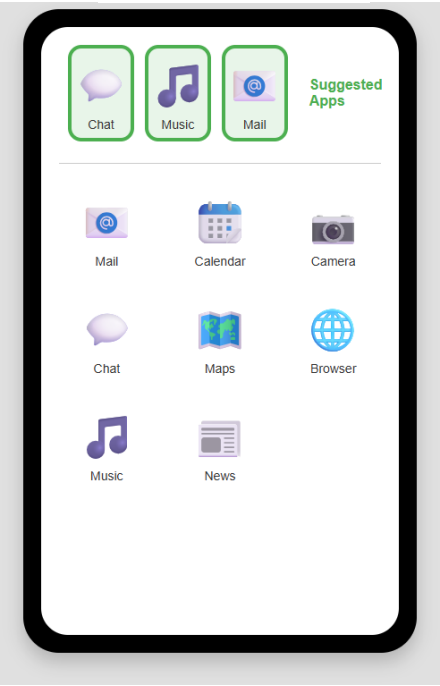
9



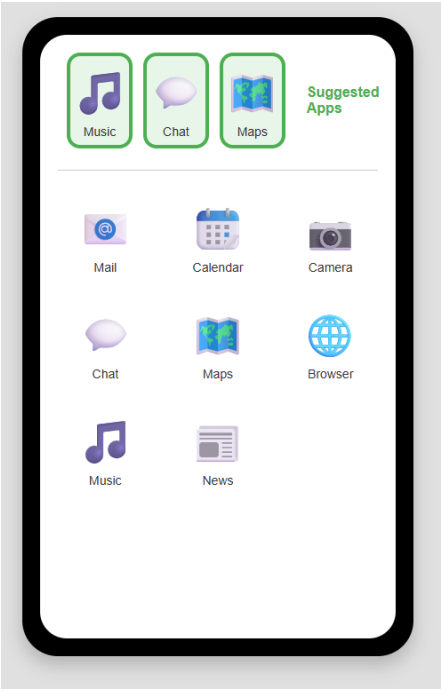
13



14



16



Takeaways

Probabilistic thinking in user modelling and adaptive UIs

- Probabilistic models allow us to **work with uncertainty** in user behaviour
  (also see the principles from last week)

- We can **combine multiple sources of evidence,**
  even if they have different data types and units;
  e.g. current app + time of day

- We **don't need to hard-code rules**
  e.g. "If A then B - but if also C then rather D"
  → Complex rules can emerge from probabilistic reasoning

- We can **achieve a lot in the UI with rather simple data & models**
  e.g. we just counted app transitions + regression on timestamps

- We can **inspect probabilities to interpret system** behaviour
  e.g. it was rather easy to inspect the impact of adding time
  → We might also use the probabilities to explain adaptation to the users

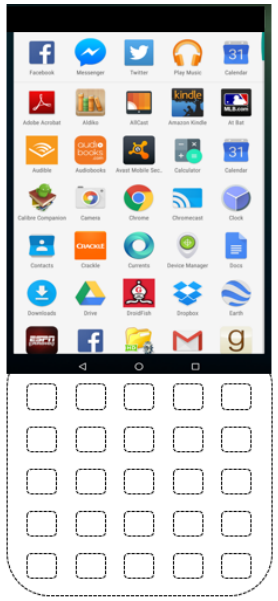
Another approach: Modelling grid search/selection

To add shortcuts for apps that would take the longest to find

Navigation model
(scrolling, either from top or bottom; e.g. based on grid row and app name)

Visual search model
(linear scan; e.g. based on distance from centre column)

No scrolling needed for these targets



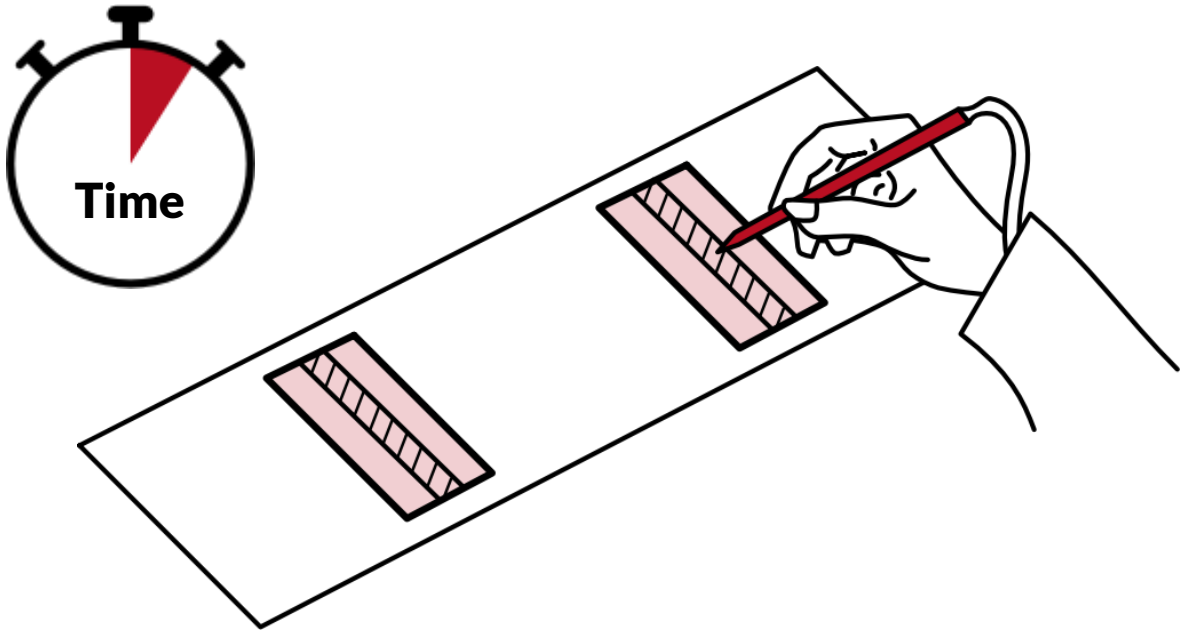
Scrolling needed for these targets

$$T_i = T_{nav} + T_{vs} + T_{point}$$

Pointing model
(based on Fitts' Law)

→ Use this model to decide which app shortcuts to display in the recommendation bar at the top (e.g. apps that would take the longest to select from the grid)

[Pfeuffer and Li, 2018]



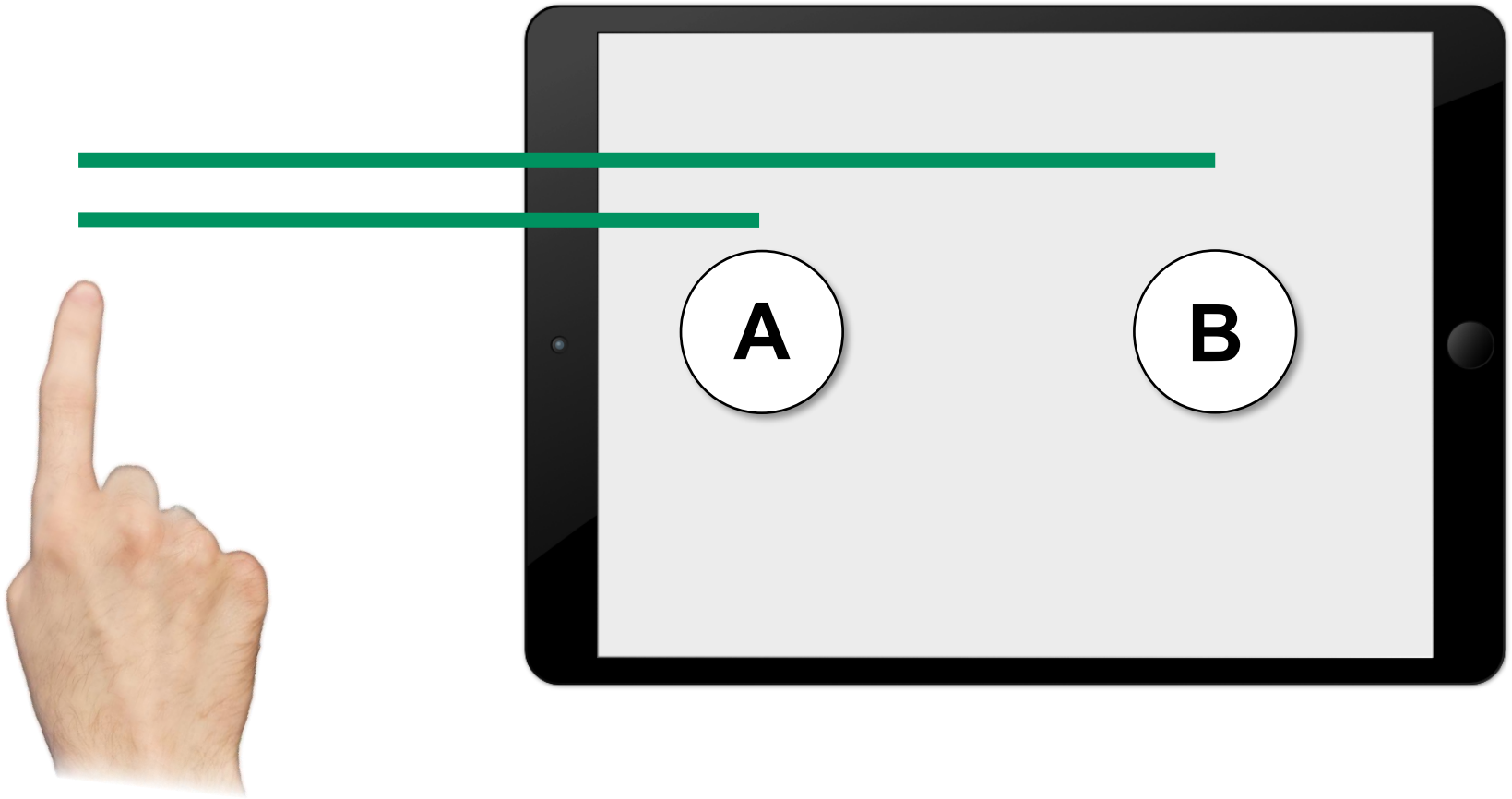
Case study 2: Fitts' Law

Data: User's targeting times; target size + distance

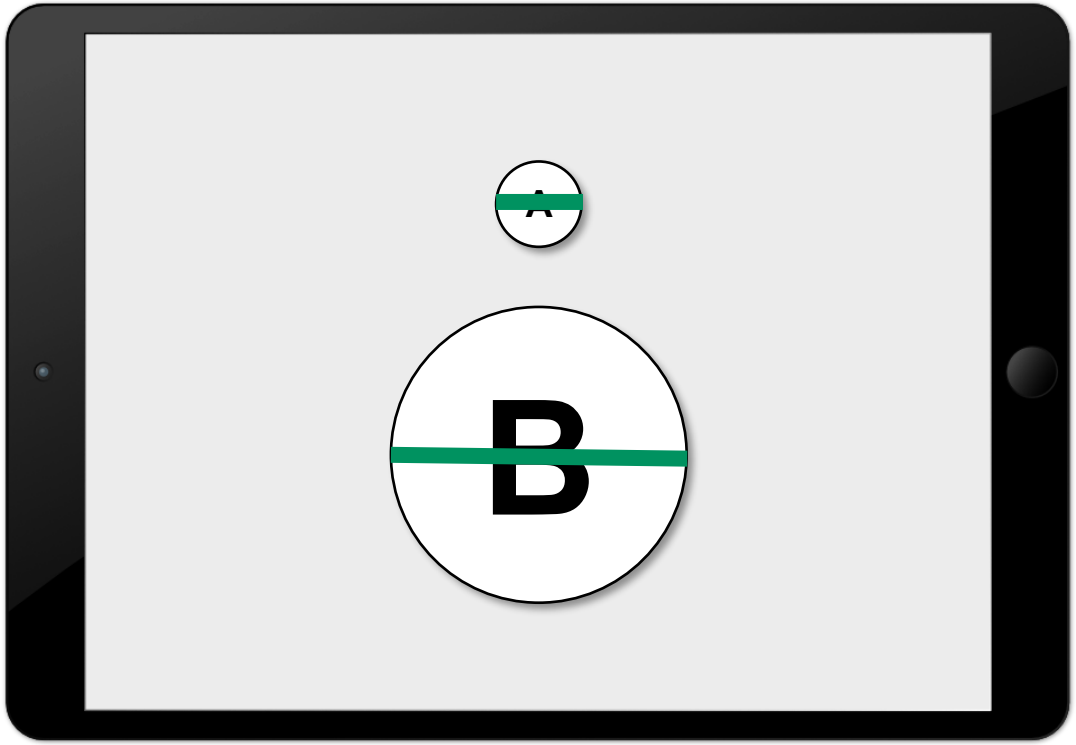
Model: Regression

Application: Estimate times, e.g. as component of other models/adaptations

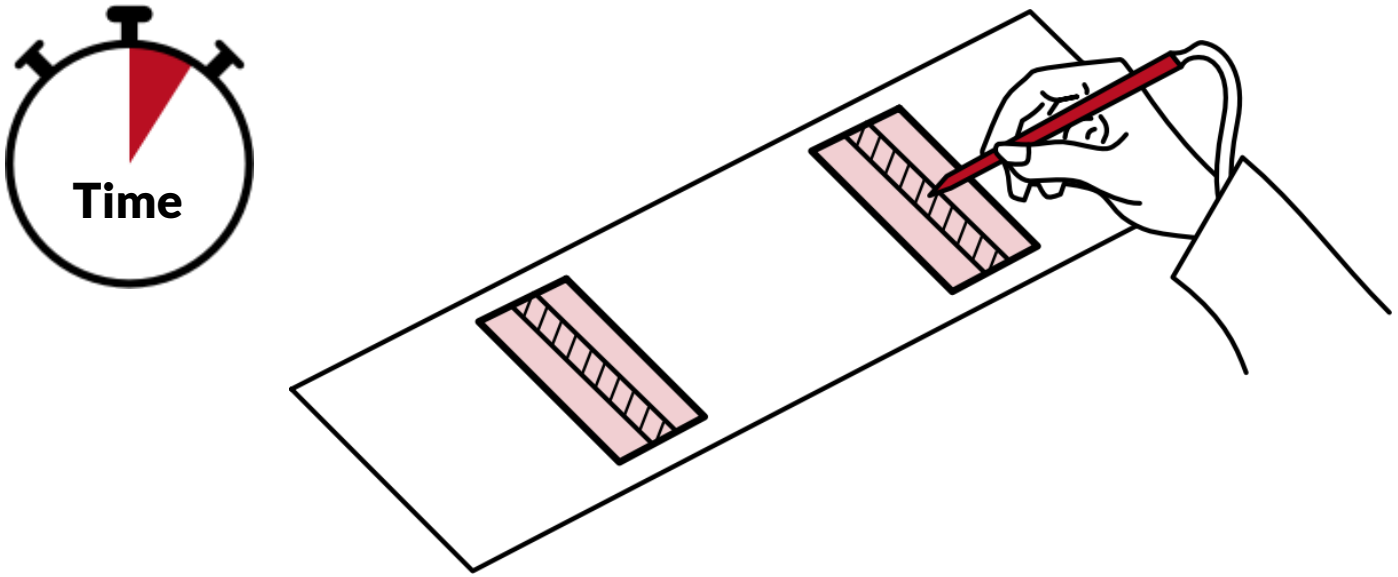
What‘s faster to reach here – A or B?



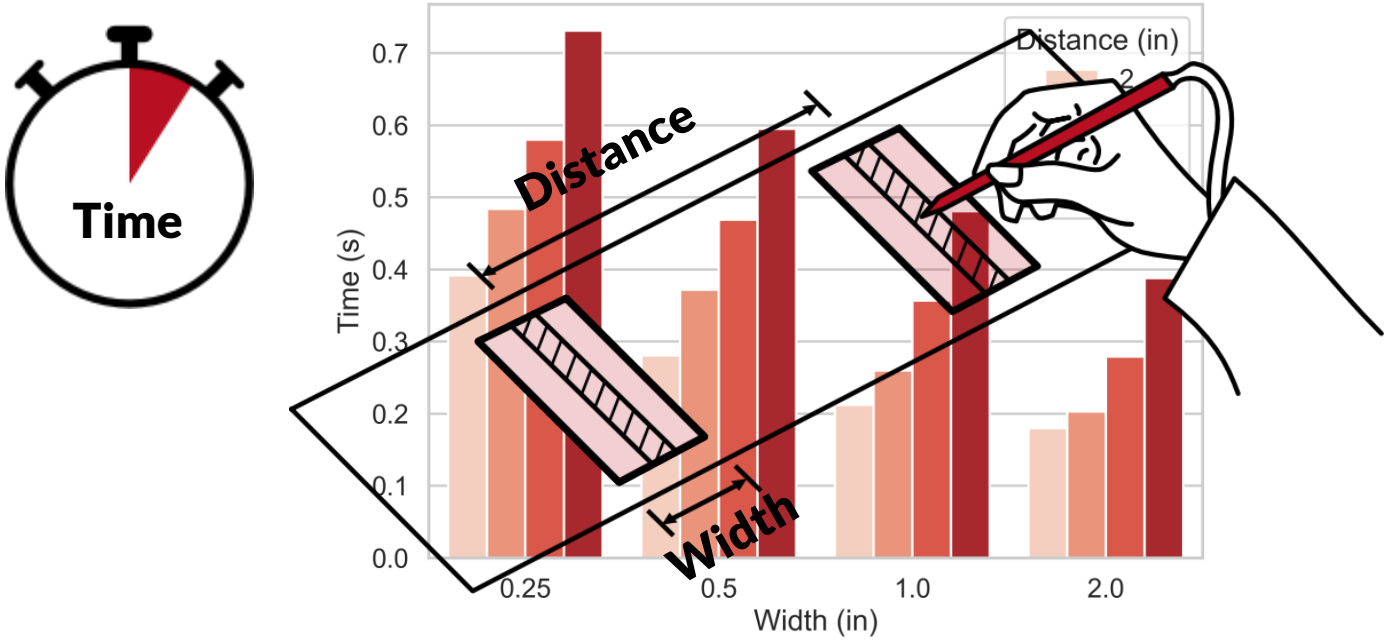
What‘s faster to reach here – A or B?



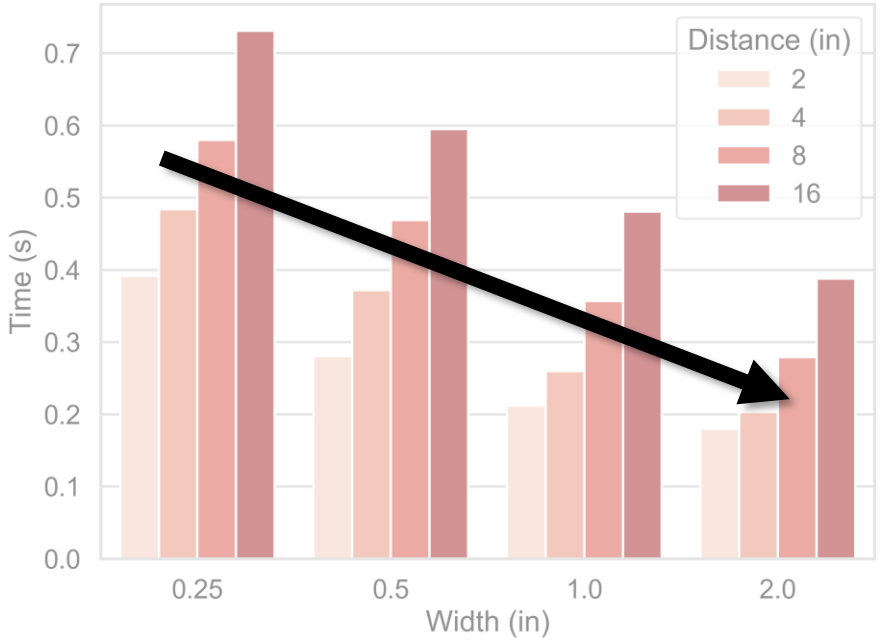
Fitts‘ Experiment (1954)



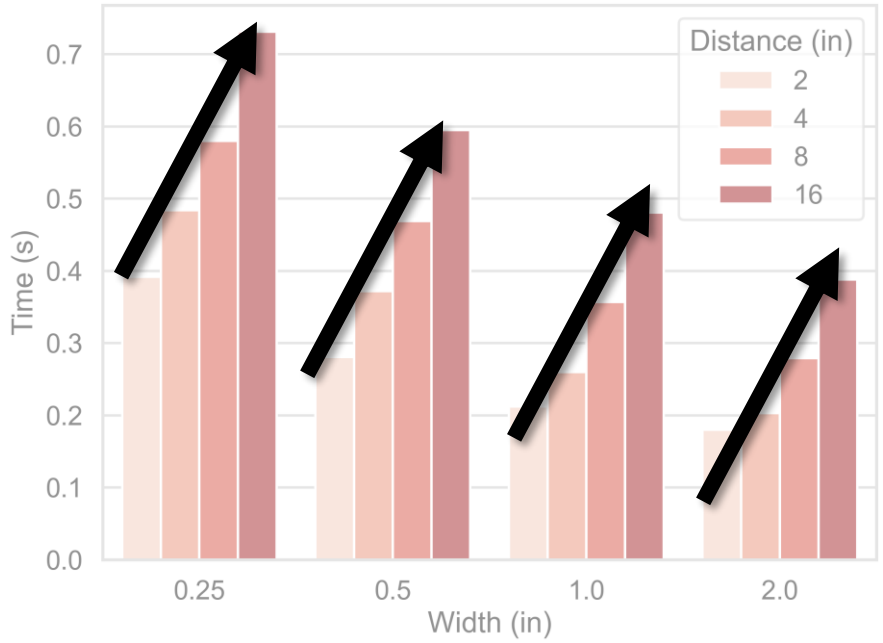
Fitts‘ Experiment (1954)



Trend 1: Larger target → faster



Trend 2: Longer distance → slower



Fitts‘ Law

Movement time to reach a target
depends on target distance und size

$$MT = a + b \log_2 \left(\frac{2D}{W} \right)$$

Constants,
depend on input device / method

ID: Index of Difficulty

$$MT = a + b \log_2 \left(\frac{2D}{W} \right)$$

Units:

D, W: e.g. cm

ID: bits

a: seconds

b: seconds per bit

- Describes the difficulty of the targeting task
- Independent of the input device / method

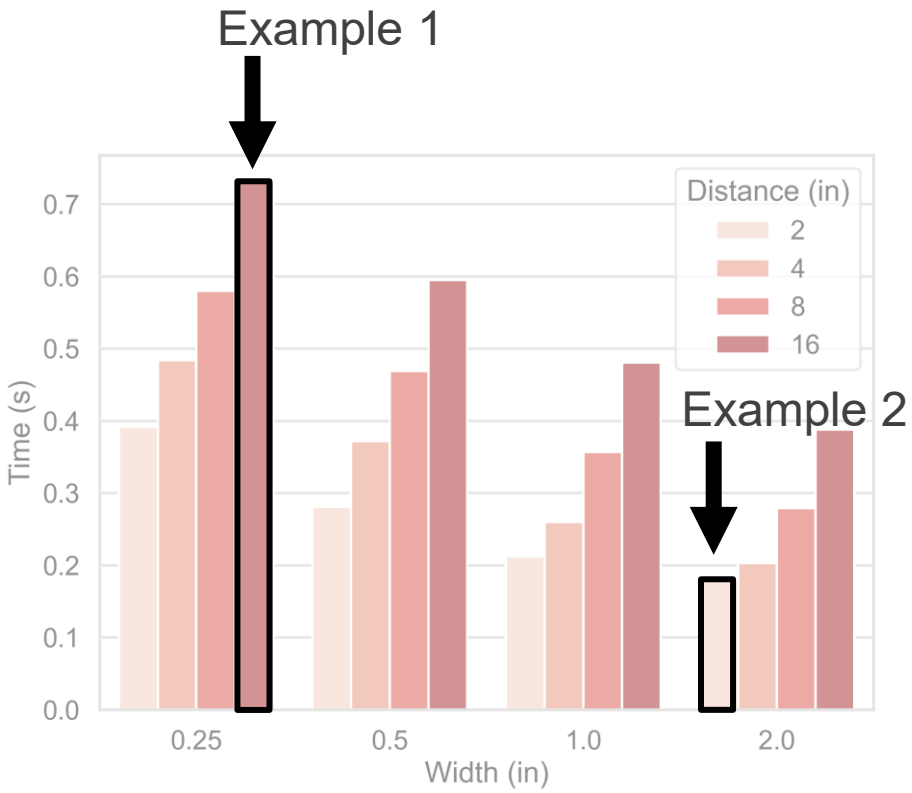
Example IDs

Example 1: W=0.25, D=16

$$ID = \log_2\left(\frac{2 \cdot 16}{0.25}\right) = 7\,\text{bits}$$

Example 2: W=2, D=2

$$ID = \log_2\left(\frac{2 \cdot 2}{2}\right) = 1\,\text{bit}$$

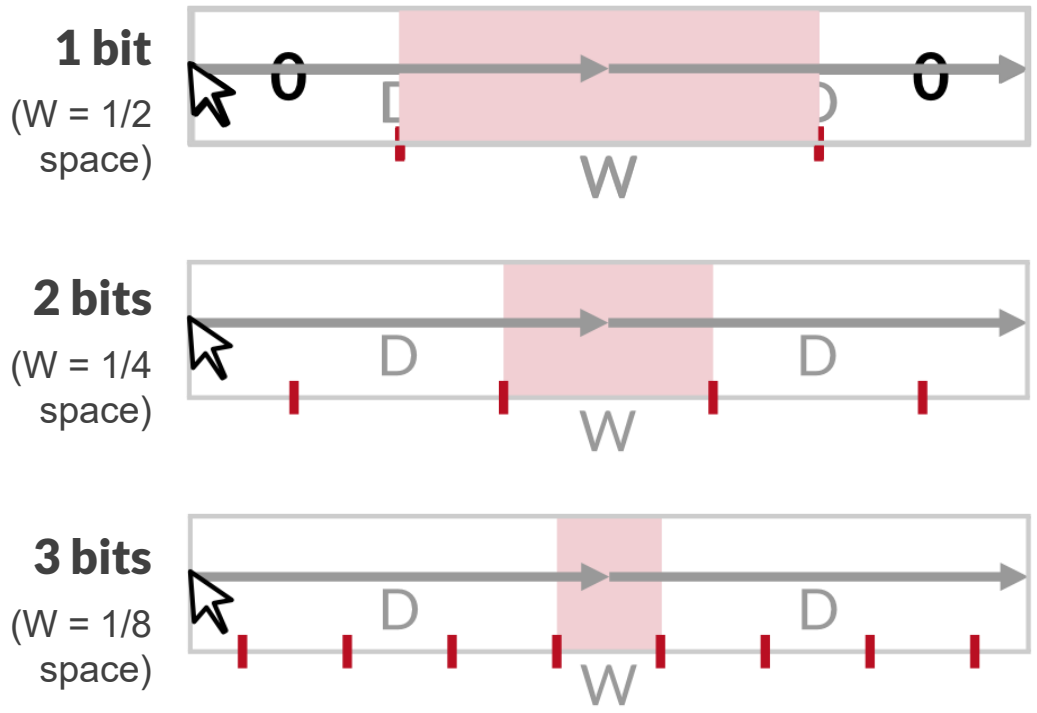


Why bits?

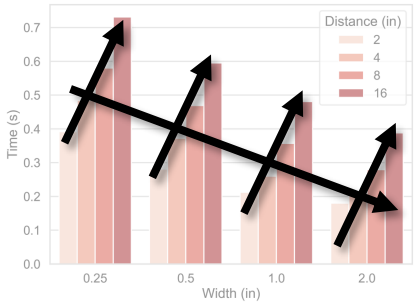
Intuition:

More bits

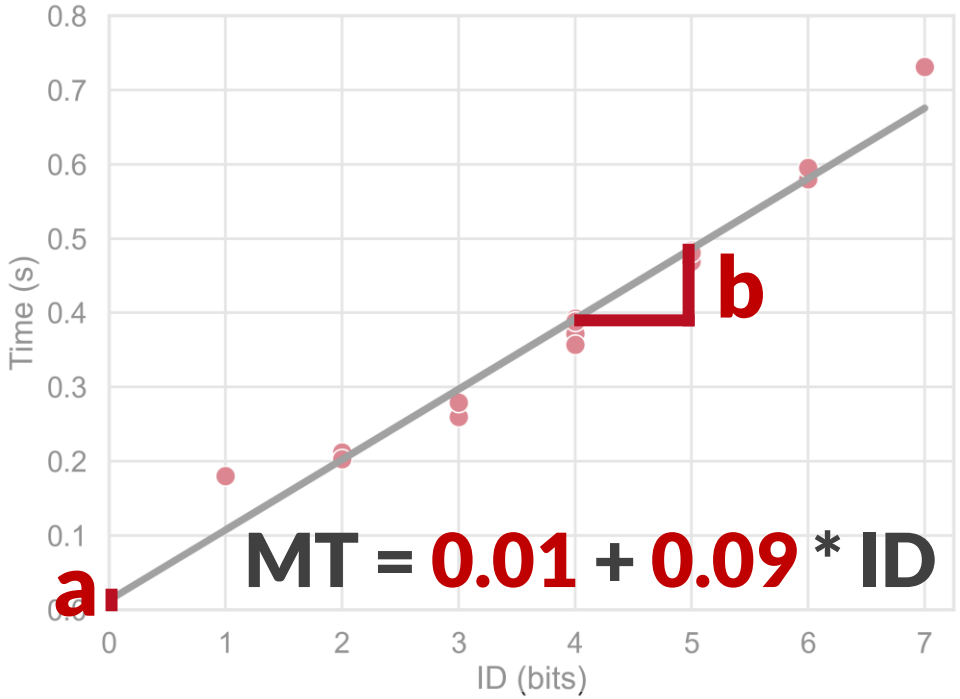
= more possible values
= more finegrained grid in space
= more accurate targeting



Regression Model



$$ID = \log_2\left(\frac{2D}{W}\right)$$



Variants & Extensions

- **Shannon form**
  MacKenzie: Analogy to information theory (Shannon-Hartley Theorem)

$$ID = \log_2\left(1 + \frac{D}{W}\right)$$

Signal

Noise

Instead of

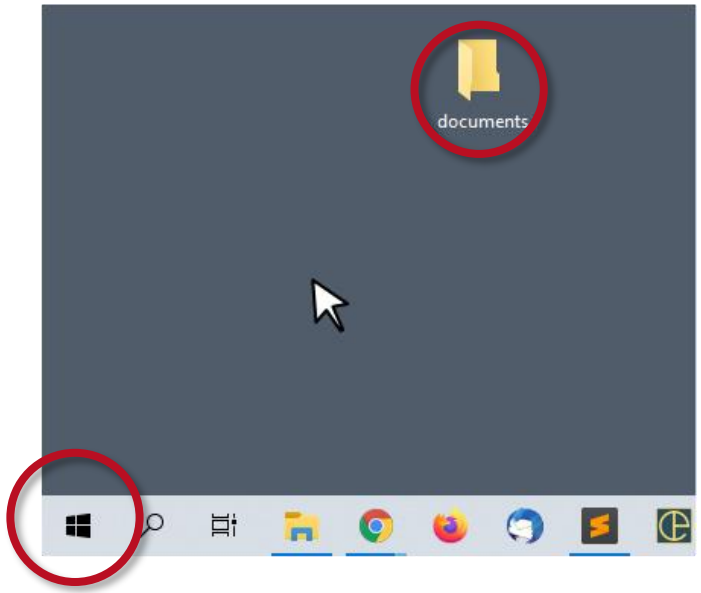
$$ID = \log_2\left(\frac{2D}{W}\right)$$

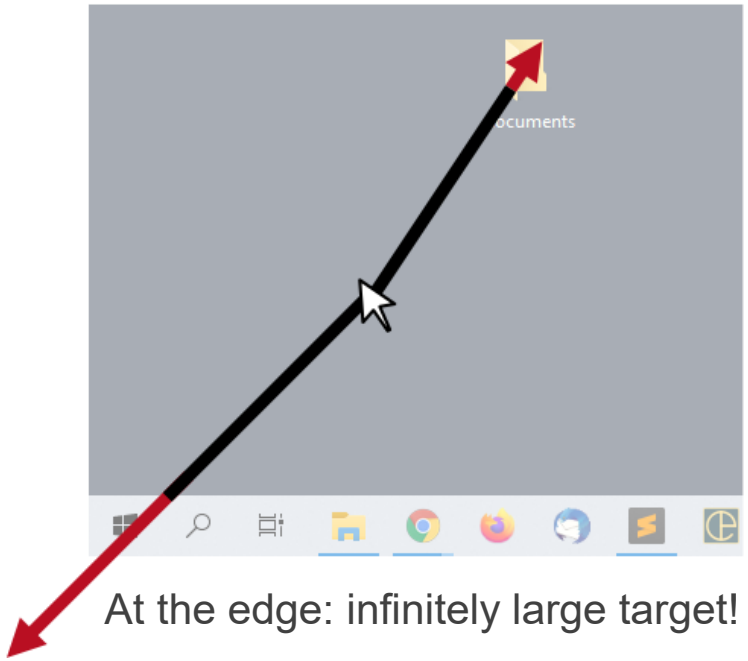
- **Extension for 2D targets**
  e.g. measure W in direction of movement

Example applications of Fitts‘ Law

Analysing user interfaces

Which target is faster to reach?



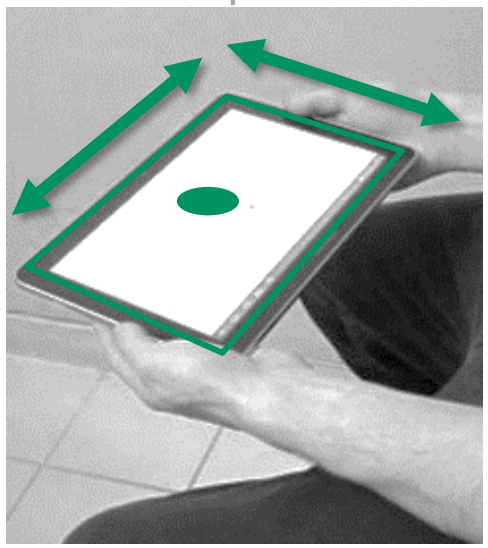


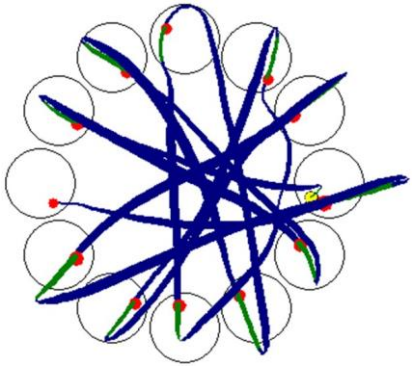
At the edge: infinitely large target!

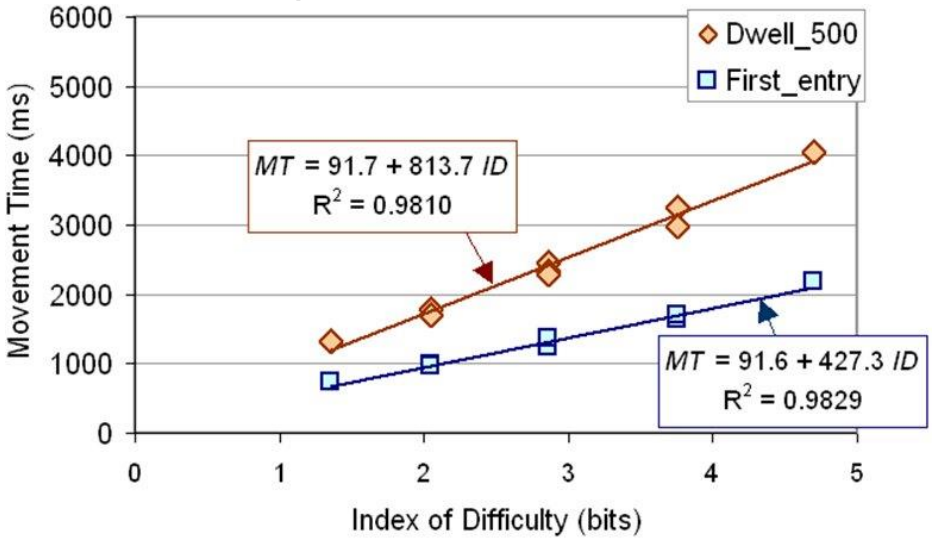
Example applications of Fitts‘ Law

Comparing interaction techniques

Move cursor via tablet tilt; two variations compared
with data from a targeting task
using regression models (i.e. Fitts‘ Law)





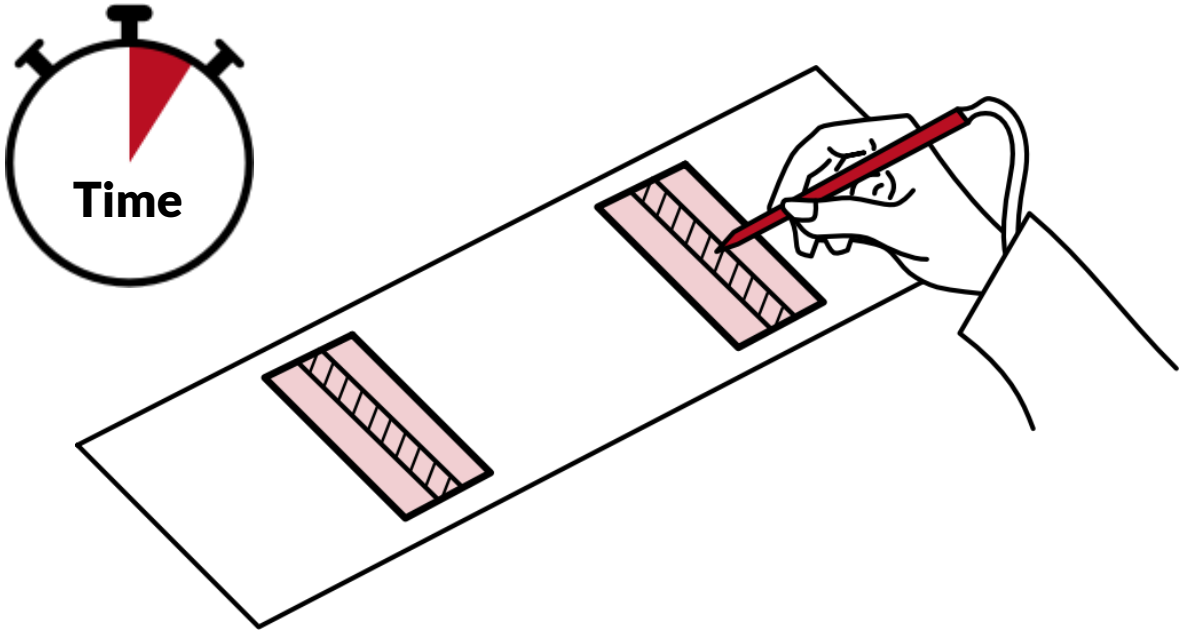


[MacKenzie and Teather, 2012]

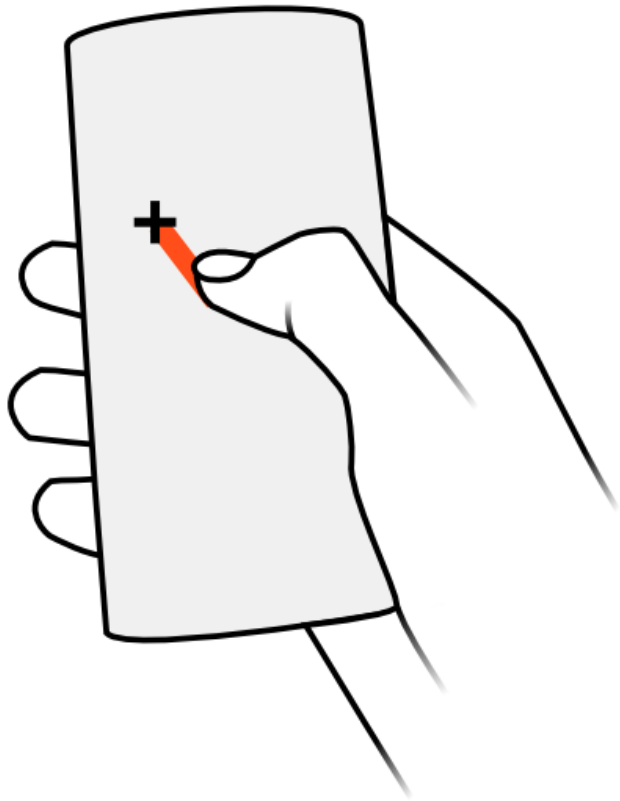
Takeaways

Fitts‘ Law is…

- A **model** of human movement
  (targeting depending on distance and size)
- An **application** of linear regression
  (predict movement time given task difficulty)
- A useful **tool** in HCI
  e.g. to compare UIs / interactions / devices
- A **component** of other models/adaptions
  e.g. anywhere where we want to predict
  how much time it takes the user to point at sth.



$$MT = a + b \log_2 \left(\frac{2D}{W} \right)$$



Case study 3: Touch offsets

Data: User's targeting offsets

Model: Regression

Application: Adapt touch interpretation to improve accuracy, infer input style

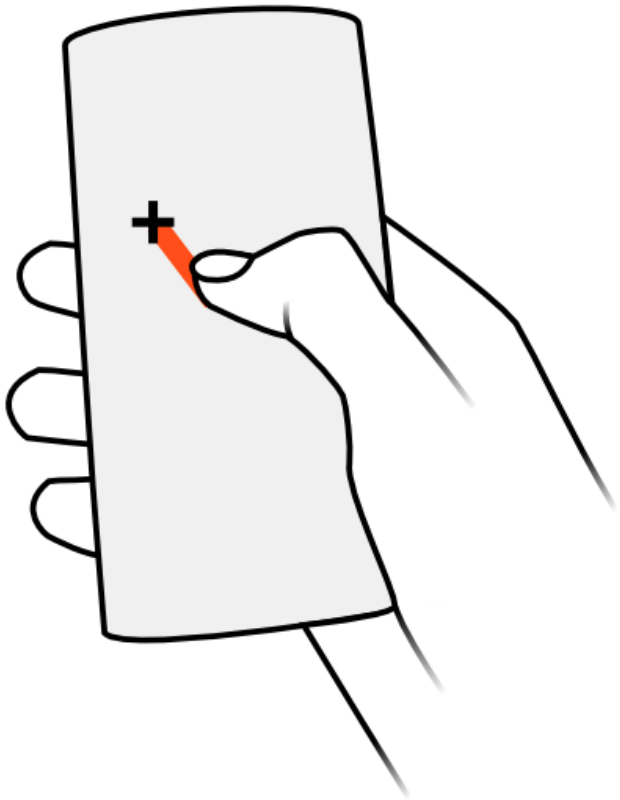
Targeting errors

Why do we not always hit buttons perfectly?

- Fingertip is not one pixel and has softness
- Finger occludes target

„fat finger problem“

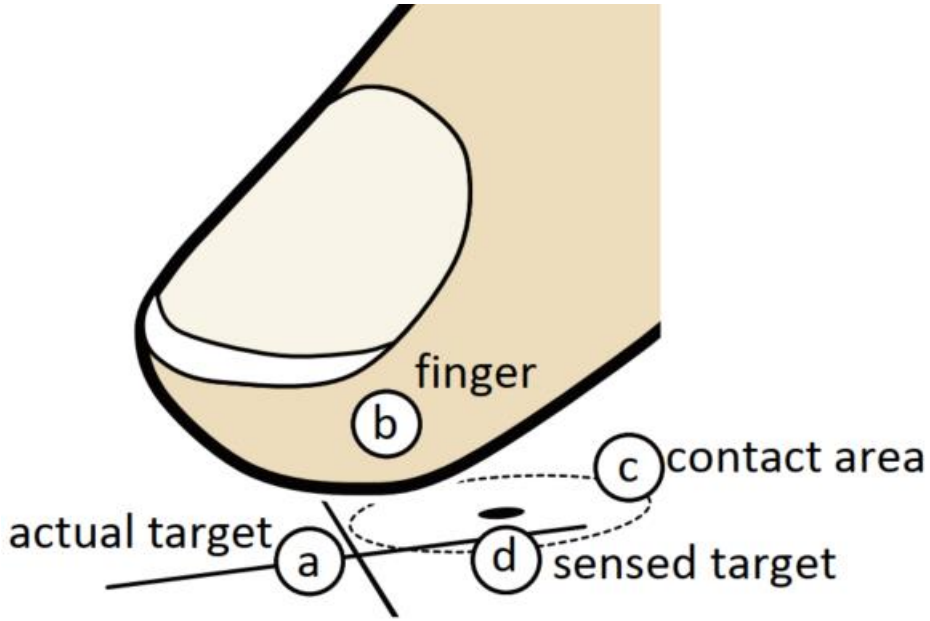
- Parallax (visual axis: eye-finger-target)
- Flat screen, no haptic cues
- Body movement (e.g. walking)
- External movement (e.g. train)
- …

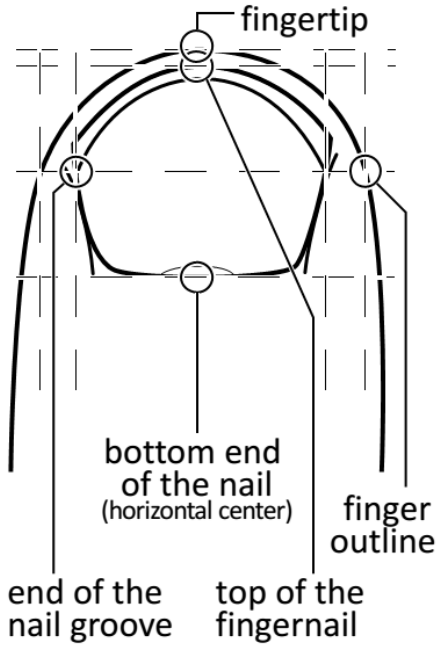


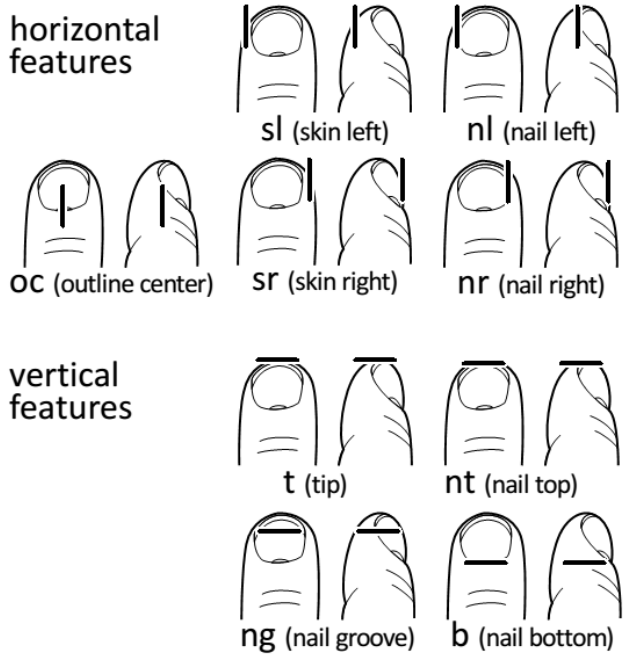
A closer look at finger targeting

Holz & Baudisch, 2011

People use different visual features to align finger and target







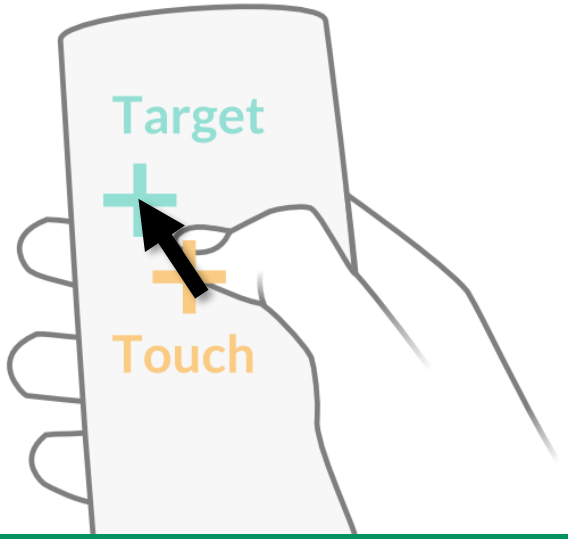
Overcoming targeting errors

Observation from previous slides:
Touch accuracy **depends on human factors**, not only on hardware
(e.g. higher resolution screen wouldn‘t not help)

→ Motivates **models** of input behaviour and **adaptation**

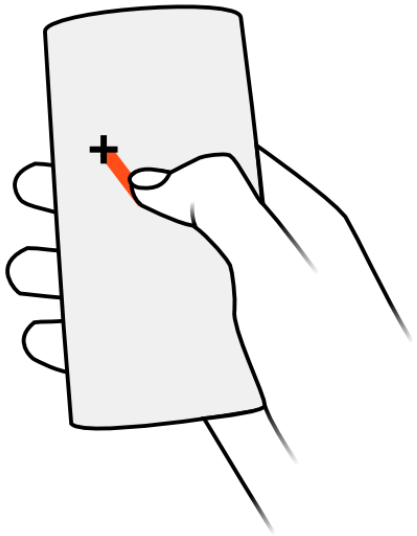
Q: Can you think of a very simple targeting model?

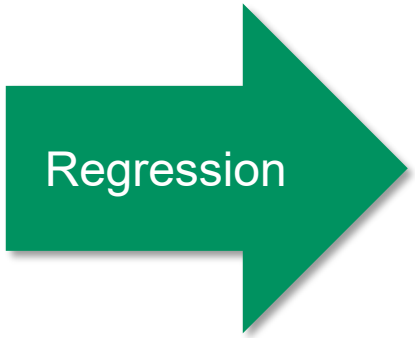
„  “ = 2D vector = model of user behaviour (e.g. average offset)



Model

Learning touch offset patterns from touch points

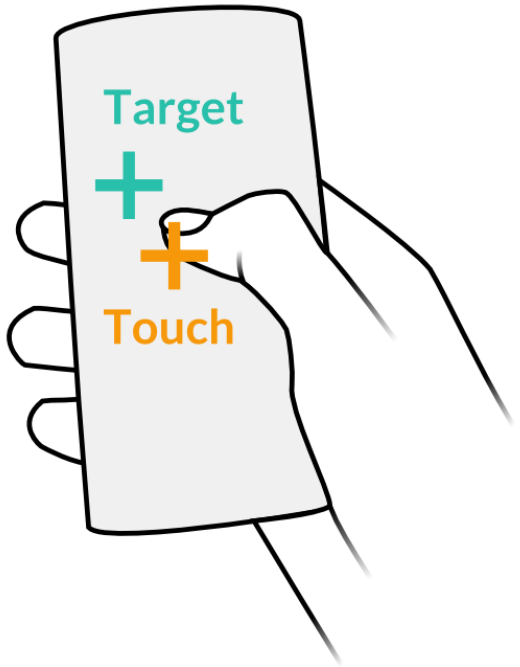




[Buschek et al., 2015]

Applying the model

To correct touch input interpretation - and improve accuracy



[Buschek et al., 2015]

Activity

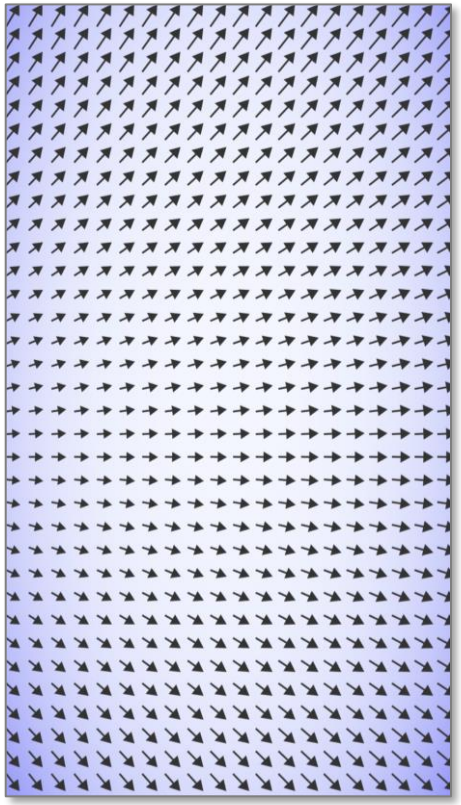
Check your own touch offset pattern

- Pick a hand posture
  (e.g. your usual way of holding your phone)
- **Hit the 16 targets** without changing the posture
- **Look at your pattern**
  – any interesting observations and insights?

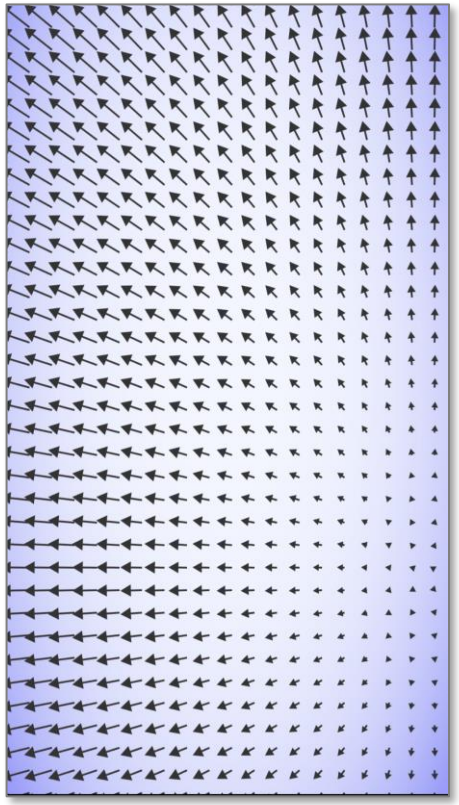
Note: Arrows point from touch to target

Examples

Left thumb



Right thumb



They look **different**…

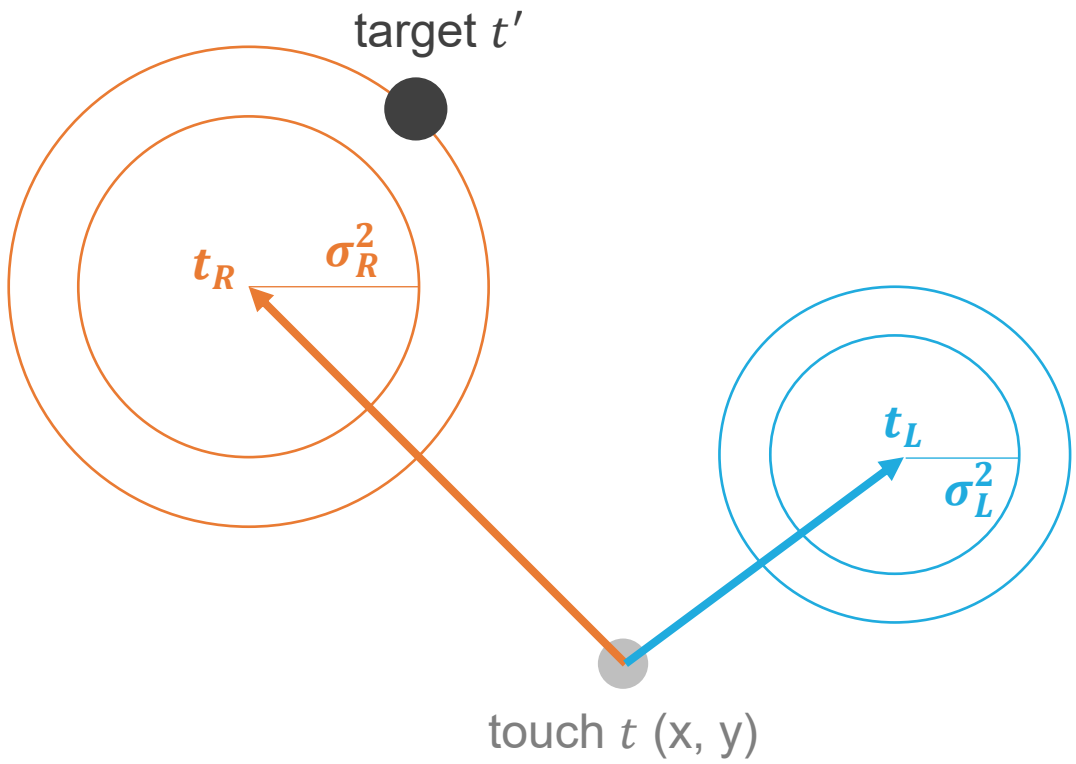
Could we use these two models to **infer** the hand posture (e.g. to adapt a UI layout)?

Probabilistic inference

With our touch models

- Probability of the target, assuming it was the left hand:
  $p(t'|L) = p(t'|t_L, \sigma_L^2) = N(t_L, \sigma_L^2)$
- Probability of the target, assuming it was the right hand:
  $p(t'|R) = p(t'|t_R, \sigma_R^2) = N(t_R, \sigma_R^2)$
- Prior probability of the left hand: $p(L) = 0.5$
- Prior probability of the right hand: $p(R) = 0.5$
- **Inference:**
  - $p(L|t') = \frac{p(t'|L)p(L)}{p(t'|L)p(L)+p(t'|R)p(R)}$
  - $p(R|t') = \frac{p(t'|R)p(R)}{p(t'|L)p(L)+p(t'|R)p(R)}$

Using Bayes' theorem (more next session)



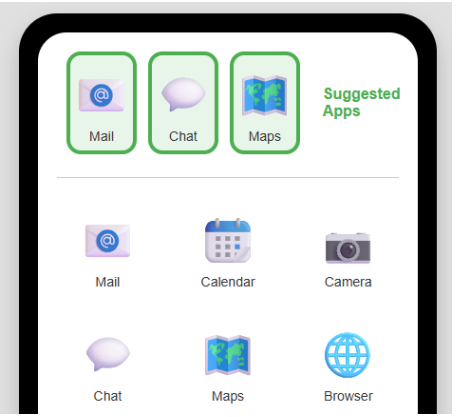
Activity

Hand posture inference with your offset models

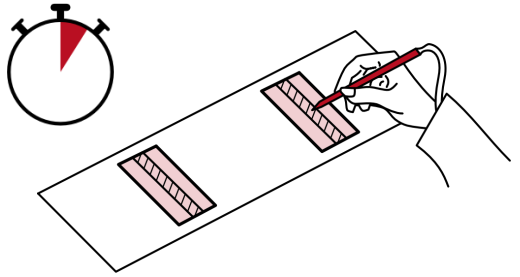
- Hit the **first 9 targets** with your **left** hand
- Hit the **next 9 targets** with your **right** hand
- Hit **further targets** to get a visualised prediction
  (tap a second time to get a new target)
- **Look at the predictions**
  – any interesting observations and insights?

Conclusion & Outlook

Summary



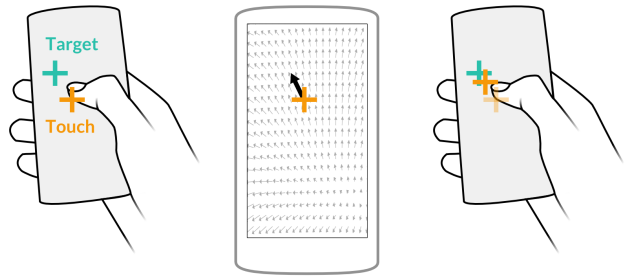
- **To adapt UIs** to the (individual) user at runtime, we typically **need to model** interaction behaviour



- Our **three case studies** covered typical “scopes” of models:
  - **Choices:** Modelling app sequences to predict next apps
  - **Time:** Modelling pointing behaviour to predict movement time
  - **Space:** Modelling touch behaviour to predict offsets

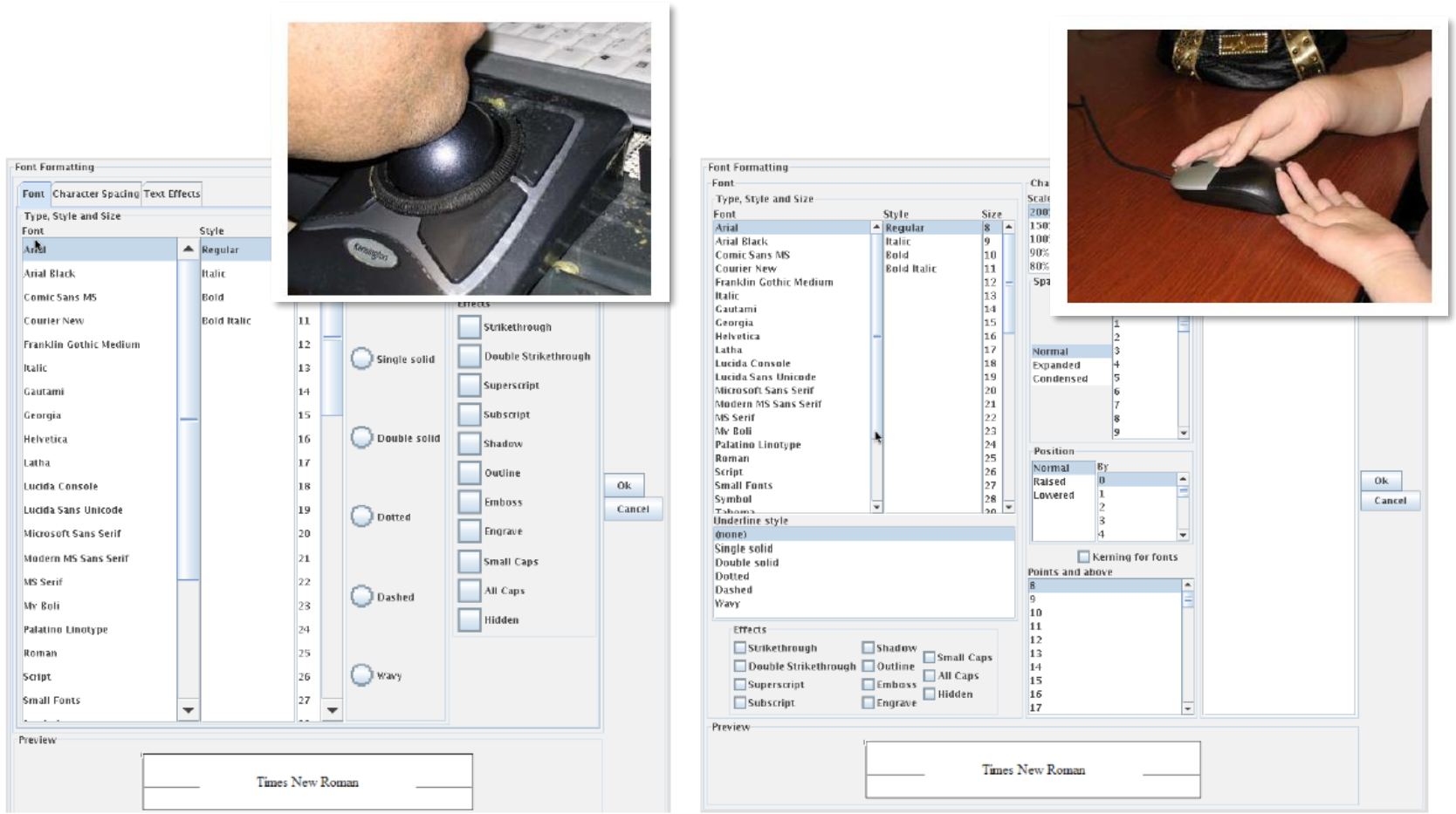
$$MT = a + b \log_2 \left(\frac{2D}{W} \right)$$

- We highlighted the value of taking a **probabilistic perspective**, since predicting user behaviour comes with **uncertainty**



Adaptation for accessibility

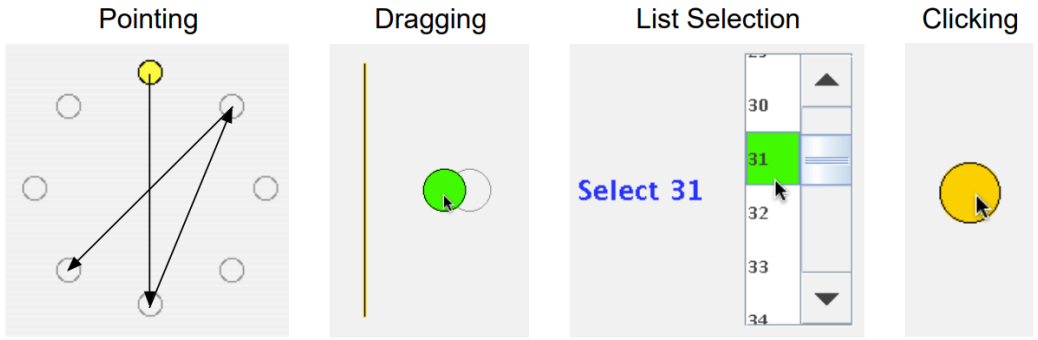
Ability-based UI optimisation

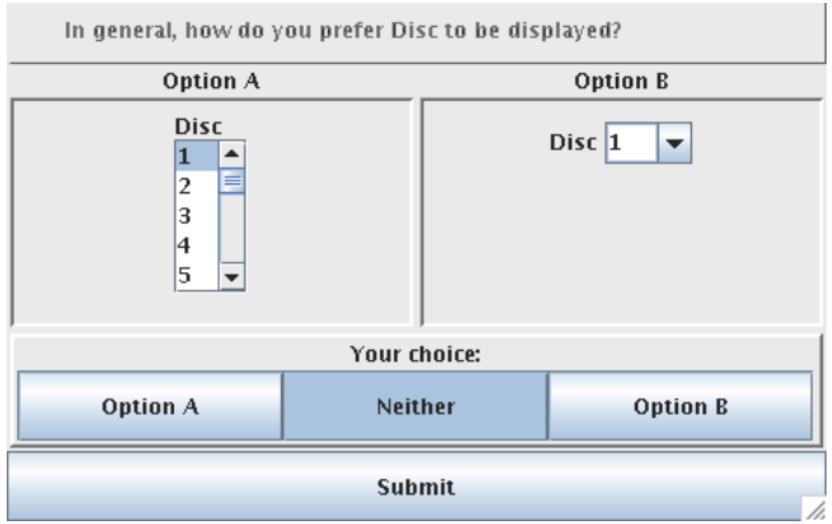


[Gajos et al., 2008]

Adaptation for accessibility

Ability-based UI optimisation





One-time task
to asses user‘s performance with different GUI elements

Alternative:
directly ask
for preference

[Gajos et al., 2008]

When is adaptation successful?

- **Adaptations have costs!** E.g. watch our for:
  - Distraction of the user
  - Re-orientation efforts required by the user
  - Change blindness (user does not notice adaptation → has outdated mental model)
  - Failure cases: Wrong adaptation makes things worse for user

- Typical **evaluations**:
  - Empirical studies with users
  - Comparison to a non-adaptive baseline system
  - Comparison of different variants of adaptations

References

References

- Browne, D. (1990). *Adaptive User Interfaces*. Elsevier.
- Buschek, D., & Alt, F. (2015). TouchML: A Machine Learning Toolkit for Modelling Spatial Touch Targeting Behaviour. *Proceedings of the 20th International Conference on Intelligent User Interfaces*, 110–114. https://doi.org/10.1145/2678025.2701381
- Christian Holz and Patrick Baudisch. 2011. Understanding touch. In Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '11). Association for Computing Machinery, New York, NY, USA, 2501–2510. https://doi.org/10.1145/1978942.1979308
- MacKenzie, I. S., & Teather, R. J. (2012). FittsTilt: the application of Fitts' law to tilt-based interaction. *In Proceedings of the 7th Nordic Conference on Human-Computer Interaction: Making Sense Through Design,* 568-577. https://doi.org/10.1145/2399016.2399103
- Pfeuffer, K., & Li, Y. (2018). Analysis and Modeling of Grid Performance on Touchscreen Mobile Devices. *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*, 1–12. https://doi.org/10.1145/3173574.3173862